

Adaptive Equalization - DSP

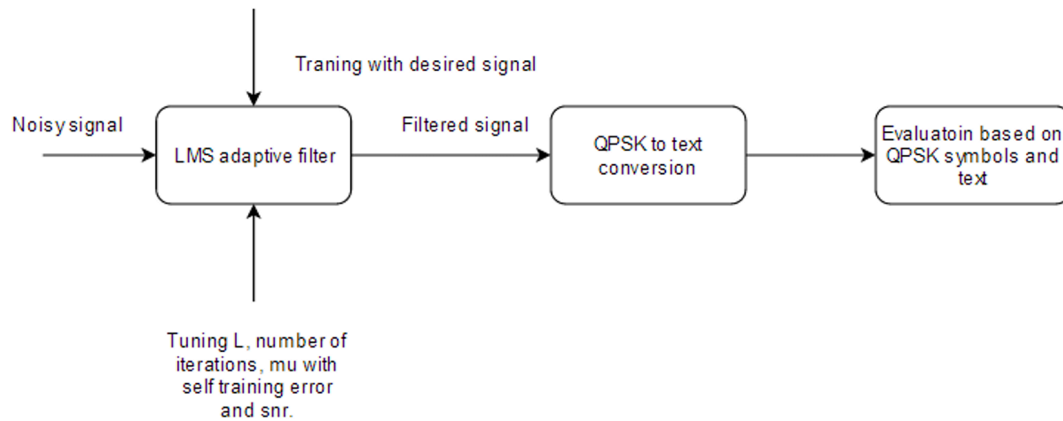


Fig 1: Block Diagram of the algorithm

Overview:

We retrieve noisy signals from the provided signals and we also generate lower SNR signals in order to challenge our design. To design adaptive filter, we first train the filter with the desired signal. The desired signal is an eighty-symbol QPSK sequence which indicates all possible characters in our ground truth text file.

After we train the desired signal, we run LMS algorithm over the noisy signal and evaluate the performance with the self-test training error as MSE and SNR ratio of the generated QPSK sequence.

These two metrics are only used for determining an ambiguous range of the idea order of our filter.

Then the filtered signal which is a sequence of QPSK signal is converted into text sequence. Eventually, the text sequence will be used to evaluate the ultimate

performance of our algorithm by counting the error rate of text characters and QPSK symbols.

Description:

Filter Order:

- In my `lms_decode.m` function, I run through 2 to 40 possible orders of our filter and evaluate the immediate performance with the MSE error and SNR ratio of them.
- Order that generates smaller MSE error and SNR ratio will be displayed. This gives me a range of orders I need to tune for my adaptive filter. Mostly I get order 2 to 11 for different kinds of signals. Then we manually tune these order numbers by comparing the ultimate error rate. After extensive experiments, I decided to use order 6 and 8 for my adaptive filter. Order 8 is mostly used for time-varying noisy signals based on our results. This is because the time-varying signals need more information when we use longer orders in optimal situation.
- If we use order 11 however, the error rate increases, at 10 dB level, by 3%. This tells us that, there is a local minimal for optimizing the number of orders. Order 6 is used for time-invariant signals, because taking many time-invariant signals in one iteration will degrade the performance of our filter. Smaller order is also not acceptable because they do not carry enough information to train the proper model. With lower orders, only-noisy signals at 15db sometimes will also have error which could only occur at 5-10 dB with the proper order for our filter. The performance of using order 6 and 8 is negligible if the number of iterations is fixed. Therefore, we use order 6 for general use.

```
[~,~,error,snr] = lms_filter(x,2,d);  
L = 2;  
E = error;  
SNR_QPSK = snr;  
for i = 3:40;  
    [~,~,error,snr] = lms_filter(x,i,d);  
    E = [E error];  
    SNR_QPSK = [SNR_QPSK,snr];  
end  
[min_E,order_desired_E] = min(E);  
[max_SNR,order_desired_SNR] = max(SNR_QPSK);
```

```
[~,~,error,snr] = lms_filter(x,2,d);
```

Runs the LMS adaptive filter with filter order 2 using input signal x and desired signal d. The first two outputs are ignored, while the error and SNR values are stored.

```
L = 2;
```

Stores the initial filter order being tested.

```
E = error;  
SNR_QPSK = snr;
```

Initialize arrays to store error and SNR values for different filter orders. The values corresponding to order 2 are stored first.

```
for i = 3:40
```

Loops through filter orders from 3 to 40 to evaluate filter performance for different numbers of taps.

```
[~,~,error,snr] = lms_filter(x,i,d);
```

Runs the LMS filter with the current filter order i and computes the corresponding error and SNR.

```
E = [E error];
```

Appends the error obtained for the current filter order to the error array E.

```
SNR_QPSK = [SNR_QPSK,snr];
```

Appends the SNR obtained for the current filter order to the SNR_QPSK array.

```
end
```

Ends the loop after testing all filter orders from 2 to 40.

```
[min_E,order_desired_E] = min(E);
```

Finds the minimum error among all tested filter orders and returns:

- min_E: the minimum error value
- order_desired_E: index where the minimum error occurs

```
[max_SNR,order_desired_SNR] = max(SNR_QPSK);
```

Finds the maximum SNR among all tested filter orders and returns:

- max_SNR: the maximum SNR value
 - order_desired_SNR: index where the maximum SNR occurs
-

Filter initiation:

Filter Initialization

The LMS filter coefficients must be initialized before adaptation begins. Three initialization methods were tested:

1. All zeros
2. All ones
3. Random values

Experimental observations showed that initializing with zeros and ones produced nearly identical results, while random initialization led to less unstable performance and convergence.

Based on multiple trials, zero initialization provided a small improvement (approximately 1–2%) in overall decoding accuracy. Therefore, the filter coefficients were initialized with zeros.

```
h = zeros(L,1);
```

Creates an $L \times 1$ column vector of zeros and uses it as the initial filter coefficient vector.

```
% h = 2*(rand(L,1)-0.5)*(2/L)
```

(Alternative method, commented out)

Generates random filter coefficients in the range $[-2/L, 2/L]$. This approach was tested but resulted in more unstable performance, so it was not used in the final implementation.

Adaptive Constant μ :

I have tried to use both regular μ and normalized μ for my filter. The benefit of using normalized μ is that we do not have to know SNR of our signal beforehand. However, in this case, we do know the noise will always be complex zero mean unit variance Gaussian noise. Therefore we can generate a sample noise signal to calculate the SNR ratio of our noisy signal. This tends out to work quite well when testing with lower SNR channel from 5-10 db. Based on my experiment results, I decide to use regular μ . However, if the noise is no longer deterministic, we might need to change to normalized μ because we do not know SNR beforehand.



The LMS algorithm uses a step size (μ) to control how quickly the filter coefficients are updated during adaptation.

```
mu_max = 1/L/(S^2/2);  
mu = mu_max/100;
```

OR

```
% mu_k = 0.5/(xk'*xk+0.001);
```

Where:

- L = filter order (number of taps)
- S = signal amplitude

| The term $(S^2/2)$ represents the approximate signal power for a QPSK signal.

OR

```
% mu_k = 0.5/(xk'*xk+0.001);
```

(Alternative method, commented out)

Computes a variable step size based on the instantaneous input signal power.

Where:

- $x_k' * x_k$ = input signal energy at the current iteration
- 0.001 = small constant added to avoid division by zero

This is similar to the Normalized LMS (NLMS) approach, where the step size automatically adjusts according to the input signal magnitude. Higher input power → smaller step size, Lower input power → larger step size. This helps maintain stable adaptation under varying signal conditions.

N:

Selection of Number of Iterations (N)

```
MSE_arr = [];
% iterations = 1000:1000:10000;
iterations = length(V);
for N = iterations
    for k = L:N
        xk = x_arr(k:-1:k-L+1);
        output = (h') * xk;
        error = d_arr(k) - output;
        % mu_k = 0.5/(xk'*xk+0.001);
        % h = h + mu_k*conj(error)*xk;
        h = h+muconj(error) * (xk);
    end
    self_test = conv(h',x);
    E = d - self_test(L:end);
    MSE = sum(abs(E).^2)/N;
    MSE_arr = [MSE_arr MSE];
end
% plot(iterations,MSE_arr)
% title('MSE vs iterations')
% xlabel('iterations')
% ylabel('MSE')
```

N represents the number of training iterations used by the LMS adaptive filter. I experimented with different values of N ranging from 1000 to 10000 and analyzed the resulting Mean Squared Error (MSE).

The MSE-versus-iterations plots showed that, for time-varying signals, the filter generally converges when the number of iterations becomes close to the signal length. Using significantly more iterations does not provide much additional improvement and may even reduce overall decoding accuracy.

For example, with an order-8 filter at 15 dB SNR, choosing N close to the signal length reduced the error rate from 6.3% to 3.5% compared to using 10000

iterations.

For time-invariant signals, however, increasing the number of iterations continues to reduce the MSE and improve performance. In one experiment, an order-6 filter at 15 dB SNR achieved an error rate of 0.13% with 4000 iterations, and additional iterations could potentially reduce the error further.

To ensure reliable performance across all scenarios, the author chose $N = 10000$. Although this slightly sacrifices accuracy for time-varying signals, it provides better robustness and near-zero error performance for time-invariant signals.

```
MSE_arr = [];
```

Creates an empty array to store MSE values computed for different iteration counts.

```
% iterations = 1000:1000:10000;
```

(Commented out)

Would generate iteration counts:
1000, 2000, 3000, ..., 10000
for experimentation.

```
iterations = length(V);
```

Sets the number of iterations equal to the length of the received signal V.

```
N = iterations;
```

Stores the selected iteration count in N.

```
for i = 1:iterations
```

Runs the LMS training process for the chosen iteration count.

```
for k = 1:N
```

Loops through the signal samples while updating the adaptive filter coefficients.

```
xk = x_arr(k:-1:k-L+1);
```

Extracts the current input vector of length L for filter processing.

```
output = (h')*xk;
```

Computes the filter output by taking the dot product of the filter coefficients h and input vector x_k .

```
error = d_arr(k) - output;
```

Calculates the difference between the desired signal and the filter output.

```
h = h + mu conj(error) (xk);
```

Updates the filter coefficients using the LMS adaptation rule.

```
end
```

Ends the LMS coefficient update loop.

```
self_test = conv(h',x);
```

Applies the trained filter to the input signal using convolution.

```
E = d - self_test(L:end);
```

Computes the final prediction error after training.

```
MSE = sum(abs(E).^2)/N;
```

Calculates the Mean Squared Error by averaging the squared magnitude of the error signal.

```
MSE_arr = [MSE_arr MSE];
```

Stores the computed MSE value.

```
end
```

Ends the iteration experiment.

```
% plot(iterations,MSE_arr)
```

(Commented out)

Would plot MSE versus number of iterations to visualize convergence.

Summary:

This section investigates how the number of LMS training iterations affects filter performance. The filter is trained, tested, and its MSE is computed for different iteration counts. Based on the convergence analysis, 10000 iterations were

selected to ensure stable and reliable performance, particularly for time-invariant signals.

Training Sequence

Purpose:

Since we know the desired signal beforehand, we can use it as a reference signal to train the LMS adaptive filter. However, the desired signal is relatively short and may not provide enough samples for the required number of training iterations.

To overcome this limitation, we repeatedly concatenate the desired signal and the corresponding noisy signal segment until their lengths become greater than or equal to the required number of training iterations (N). These extended sequences are then used for adaptive filter training.

This approach ensures that sufficient training data is available throughout the adaptation process while preserving the characteristics of the original signals.

```
x_arr = []; % d is desired signal
d_arr = [];
x = V(1:length(d));
for i = 1: ceil(N/L)
    x_arr = [x_arr;x];
    d_arr = [d_arr;d];
end
```

```
x_arr = []; % Training input sequence (noisy signal)
d_arr = []; % Training target sequence (desired signal)
```

We initialize two empty arrays that will store the repeated noisy and desired signal sequences used during training.

```
x = V(1:length(d));
```

We extract the portion of the noisy signal V that corresponds to the length of the desired signal d . This ensures that both signals contain the same number of samples.

```
for i = 1:ceil(N/L)
```

We determine how many times the sequences need to be repeated.

- **N** = Number of training iterations
- **L** = Filter order

The expression `ceil(N/L)` rounds the value up to the nearest integer, ensuring that the generated training sequences are long enough for all iterations.

```
x_arr = [x_arr; x];
```

We append another copy of the noisy signal segment to the training input array, gradually building a longer input sequence.

```
d_arr = [d_arr; d];
```

We append another copy of the desired signal to the training target array, ensuring that each noisy input sample has a corresponding desired output sample.

```
end
```

This marks the end of the sequence generation loop. After completion, both `x_arr` and `d_arr` contain sufficient samples for LMS training.

Filter Adaptation

Purpose

The LMS adaptive filter updates its coefficients iteratively to minimize the error between the desired signal and the filter output. During each iteration, the filter

computes an output, evaluates the error, and adjusts its coefficients accordingly.

The adaptation loop runs from $k = L$ to $k = N$, resulting in $N - L + 1$ coefficient updates. Starting from L ensures that enough previous samples are available to form the input vector and helps avoid filter transient effects.

As discussed earlier, we choose a relatively large value of N to ensure reliable convergence and near-perfect accuracy for time-invariant signals at high SNR values. We also use a fixed step size (μ) instead of a normalized step size because it provided more stable performance in our experiments.

```
xk = x_arr(k:-1:k-L+1);
output = (h') * xk;
error = d_arr(k) - output;
% mu_k = 0.5/(xk'*xk+0.001);
% h = h + mu_k*conj(error)*xk;
h = h+muconj(error) * (xk);
```

Code Explanation

```
xk = x_arr(k:-1:k-L+1);
```

We construct the current input vector by selecting the most recent L samples from the training input sequence. The samples are arranged in reverse order, which is the standard format used in FIR adaptive filtering.

```
output = (h')*xk;
```

We compute the filter output by taking the dot product of the current filter coefficient vector h and the input vector x_k .

```
error = d_arr(k) - output;
```

We calculate the instantaneous error by subtracting the filter output from the desired signal sample. This error indicates how far the current output is from the target value.

```
% mu_k = 0.5/(xk'*xk+0.001);
```

This is a commented-out implementation of a normalized LMS (NLMS) step size. The step size is adjusted dynamically based on the input signal power, with a small constant added to avoid division by zero.

```
% h = h + mu_k*conj(error)*xk;
```

This is the corresponding NLMS coefficient update equation, which was tested but not used in the final implementation.

```
h = h + mu*conj(error)*(xk);
```

We update the filter coefficients using the standard LMS adaptation rule. The coefficient adjustment is proportional to the error, the input vector, and the chosen step size μ .

This update gradually minimizes the error and enables the filter to learn the characteristics of the desired signal.

Applying the Trained Filter:

Purpose

Once the LMS filter coefficients have been trained, we can use the filter to recover the signal from the noisy input. This is done by applying the trained filter coefficients to the entire noisy signal.

The easiest way to do this is by performing a convolution between the trained filter coefficients and the noisy signal.

```
QPSK = conv(h',V);  
QPSK = QPSK(L:length(V));
```

Code Explanation

```
QPSK = conv(h',V);
```

We apply the trained filter to the noisy signal `V` using convolution.

- `h'` contains the final trained filter coefficients.
- `V` is the noisy input signal.
- `conv()` performs the filtering operation and produces the filtered output signal.

The resulting signal `QPSK` is our estimate of the original transmitted signal after noise reduction.

```
QPSK = QPSK(L:length(V));
```

After convolution, the beginning of the filtered signal contains some extra samples caused by the filter delay.

To remove these unwanted samples, we keep only the samples from index `L` onward.

This aligns the filtered signal with the original signal and avoids errors caused by the filter's initial transient response.

QPSK to Text Conversion

Purpose

After filtering, the output signal is still a sequence of **complex QPSK symbols**. To recover the original message, we need to convert these symbols back into binary bits and then convert those bits into text characters.

In QPSK, each symbol carries **2 bits**:

- One bit is represented by the **real part** of the symbol.
- The other bit is represented by the **imaginary part** of the symbol.

We use a threshold-based decision rule to determine whether each component corresponds to a binary 0 or 1.

```
for index = 1:length(bs)
    b1 = real(bs(index));
    b2 = imag(bs(index));

    if(b1 < -Threshold)
        b1 = '1';
    elseif(b1 > Threshold)
        b1 = '0';
    else
        display('threshold situation detected')
        b1 = '1';
    end
    if(b2 < -Threshold)
        b2 = '1';
    elseif(b2 > Threshold)
        b2 = '0';
    else
        b2 = '1';
        display('threshold situation detected')
    end
    bits = [bits b1 b2];
end
demodBn = reshape(bits, bn_length , length(bits)/bn_length);
demodBn = demodBn';
ascii = bin2dec(demodBn);
```

Code Explanation

```
for index = 1:length(bs)
```

We process each QPSK symbol one by one.

```
b1 = real(bs(index));
b2 = imag(bs(index));
```

We extract:

- **b1** → Real part of the QPSK symbol
- **b2** → Imaginary part of the QPSK symbol

Since each QPSK symbol contains two bits, both components need to be decoded separately.

```
if(b1 < -Threshold)
    b1 = '1';
elseif(b1 > Threshold)
    b1 = '0';
else
    display('threshold situation detected')
    b1 = '1';
end
```

We convert the real part into a binary value using a threshold.

- Value less than **Threshold** → bit **'1'**
- Value greater than **Threshold** → bit **'0'**
- Value close to the threshold → treated as a special case and assigned **'1'**

This decision process is known as **symbol demodulation**.

```
if(b2 < -Threshold)
    b2 = '1';
elseif(b2 > Threshold)
    b2 = '0';
else
    b2 = '1';
    display('threshold situation detected')
end
```

The same threshold decision is applied to the imaginary part to recover the second bit carried by the QPSK symbol.

```
bits = [bits b1 b2];
```

We append the two recovered bits to the bit stream.

Since each QPSK symbol carries two bits, the binary sequence grows by two bits every iteration.

```
end
```

Ends the symbol decoding loop after all QPSK symbols have been processed.

```
demodBn = reshape(bits, bn_length, length(bits)/bn_length);
```

We group the recovered bit stream into blocks of fixed length.

Each block represents the binary encoding of a single character.

```
demodBn = demodBn';
```

We transpose the matrix so that each row corresponds to one character.

This makes the binary data ready for conversion into ASCII values.

```
ascii = bin2dec(demodBn);
```

We convert each binary block into its corresponding decimal ASCII value.

For example:

```
01000001 → 65  
01000010 → 66
```

Final Step

After obtaining the ASCII values, we convert them into characters to reconstruct the original text message.

For example:

```
65 → A  
66 → B
```


The sequence of characters forms the recovered message transmitted through the QPSK communication system.

Evaluation:

Purpose

To evaluate the performance of the LMS-based QPSK decoder, we compare the recovered message with the original message stored in `gettysburg.txt`, which serves as the ground truth.

First, the original text is converted into its custom ASCII representation and corresponding QPSK symbol sequence. After decoding the received signal, we compare the recovered ASCII values and QPSK symbols against these reference values to measure the accuracy of the system.

Since the received signal contains a leading synchronization sequence at the beginning, the decoded output must be shifted before comparison. In our implementation, this offset corresponds to **85 QPSK symbols**.

```
head_signal = 85; %[0:31 4 5 asciiseq(find(~isnan(asciiseq))) 0 0 0 0 0 0],
[0:31 4 5] length is 34, 34*5/2 = 85
[QPSK_real, ascii] = file2bin(filename);
txt_real = char(ascii);
txt_test = bin2text(QPSK_test(head_signal+1-order_desired+1:end));
txt_test=txt_test(1:length(txt_real));
```

Code Explanation

```
head_signal = 85;
```

We define the number of leading QPSK symbols that must be skipped before the actual message begins.

This offset exists because the transmitted signal contains a synchronization/header sequence before the text data.

```
% [0:31 4 5 asciiseq(find(~isnan(asciiseq))) 0 0 0 0 0 0]
```

This comment describes the structure of the transmitted sequence.

The header consists of:

```
[0:31 4 5]
```

which contains 34 values.

```
% [0:31 4 5] length is 34, 34*5/2 = 85
```

Each value is converted into binary and then mapped into QPSK symbols.

Since each QPSK symbol carries 2 bits, the 34 header values occupy a total of 85 QPSK symbols. Therefore, we skip the first 85 symbols during evaluation.

```
[QPSK_real, ascii] = file2bin(filename);
```

We generate the ground-truth data from the original text file.

This function converts the text file into:

- `ascii` → Custom ASCII representation
- `QPSK_real` → Corresponding QPSK symbol sequence

These serve as the reference values for comparison.

```
txt_real = char(ascii);
```

We convert the reference ASCII values into readable text.

This represents the original message that was transmitted.

```
txt_test = bin2text(QPSK_test(head_signal+1-order_desired+1:end));
```

We convert the decoded QPSK symbols back into text.

Before conversion, we remove:

- The leading synchronization sequence (`head_signal`)
- The filter delay introduced by the adaptive filter (`order_desired - 1`)

This ensures that the recovered message is properly aligned with the original text.

```
txt_test = txt_test(1:length(txt_real));
```

We trim the decoded message so that its length matches the original message length.

This allows a fair character-by-character comparison during evaluation.

CONSTITUTION PLOT:

Noise-only-

We can achieve around 8.0% error rate give 7db SNR for noise-only signals. We use order 6 and 10000 iterations. If it is 6db, the error rate jumps up to 16.1%. The constellation plots for the "before" and "after" signals are shown below.

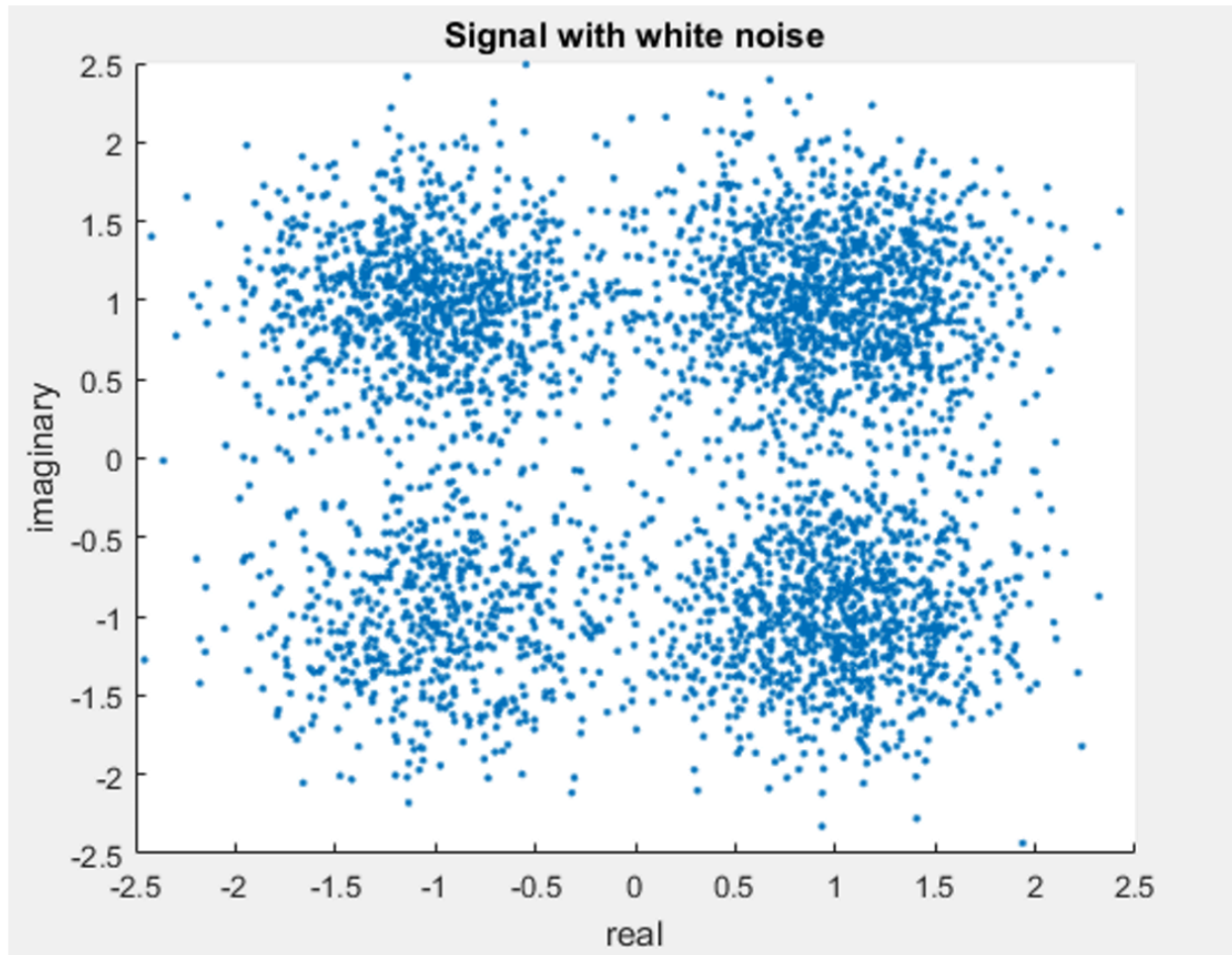


Figure 2: Constellation plot of SNR = 7db of noise-only signals before filtering.

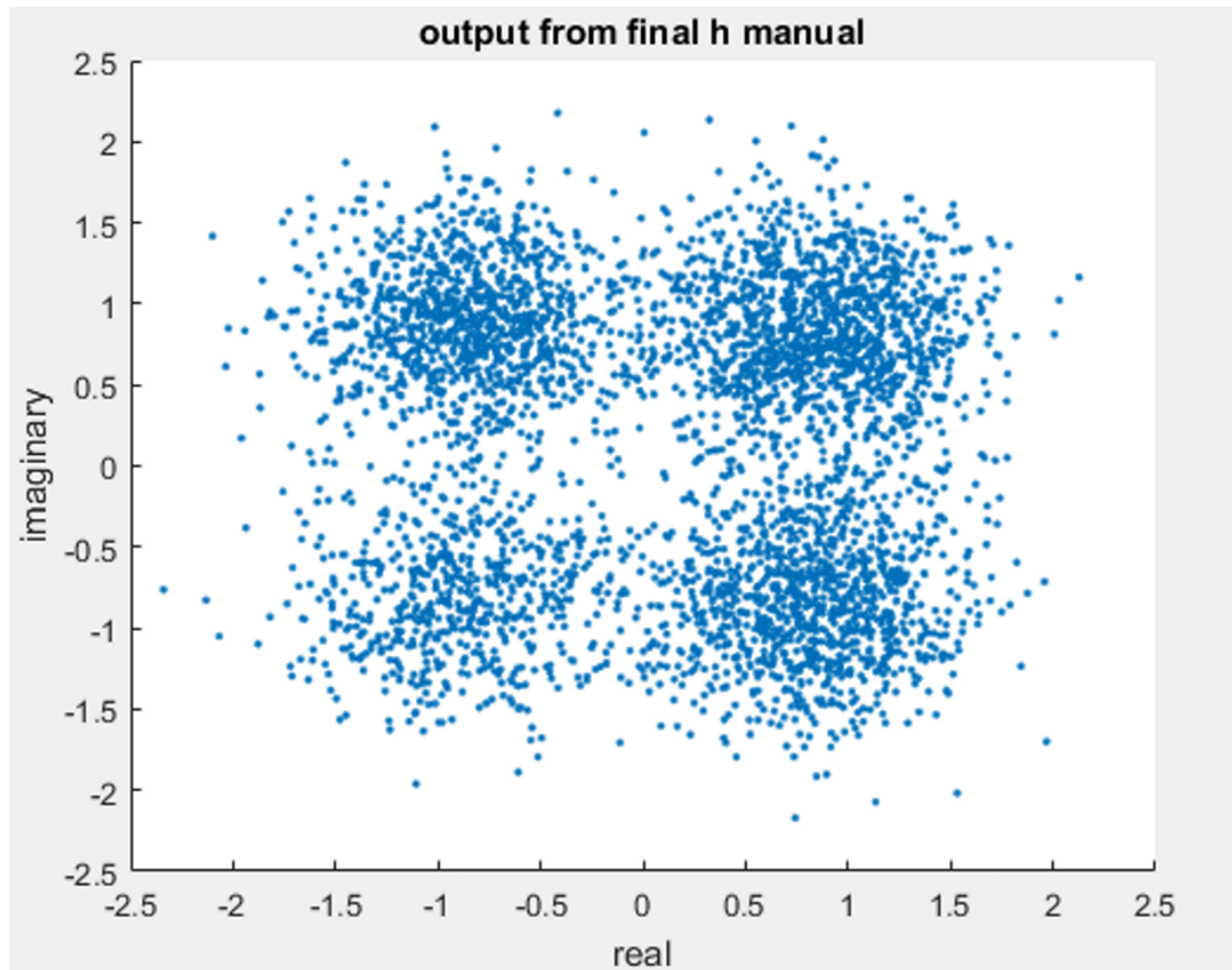


Figure 3: Constellation plot of SNR = 7db of noise-only signals after filtering



output from final h manually choose order 6
Error rate is 3.181258% over 4055 QPSK symbols
Error rate is 7.964602% over 1582 text

Time-invariant-

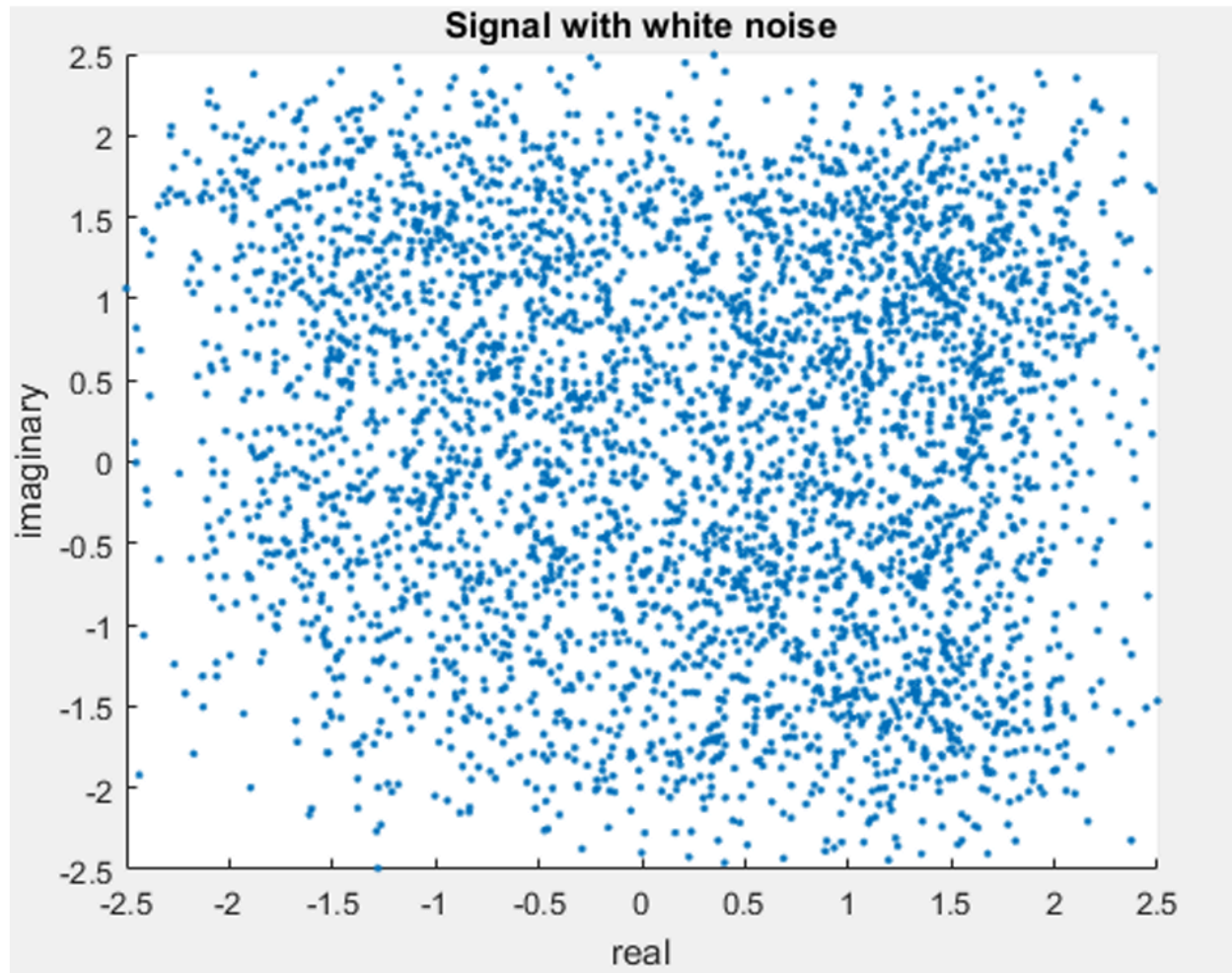


Figure 4: Constellation plot of SNR = 9db of time-invariant signals before filtering.

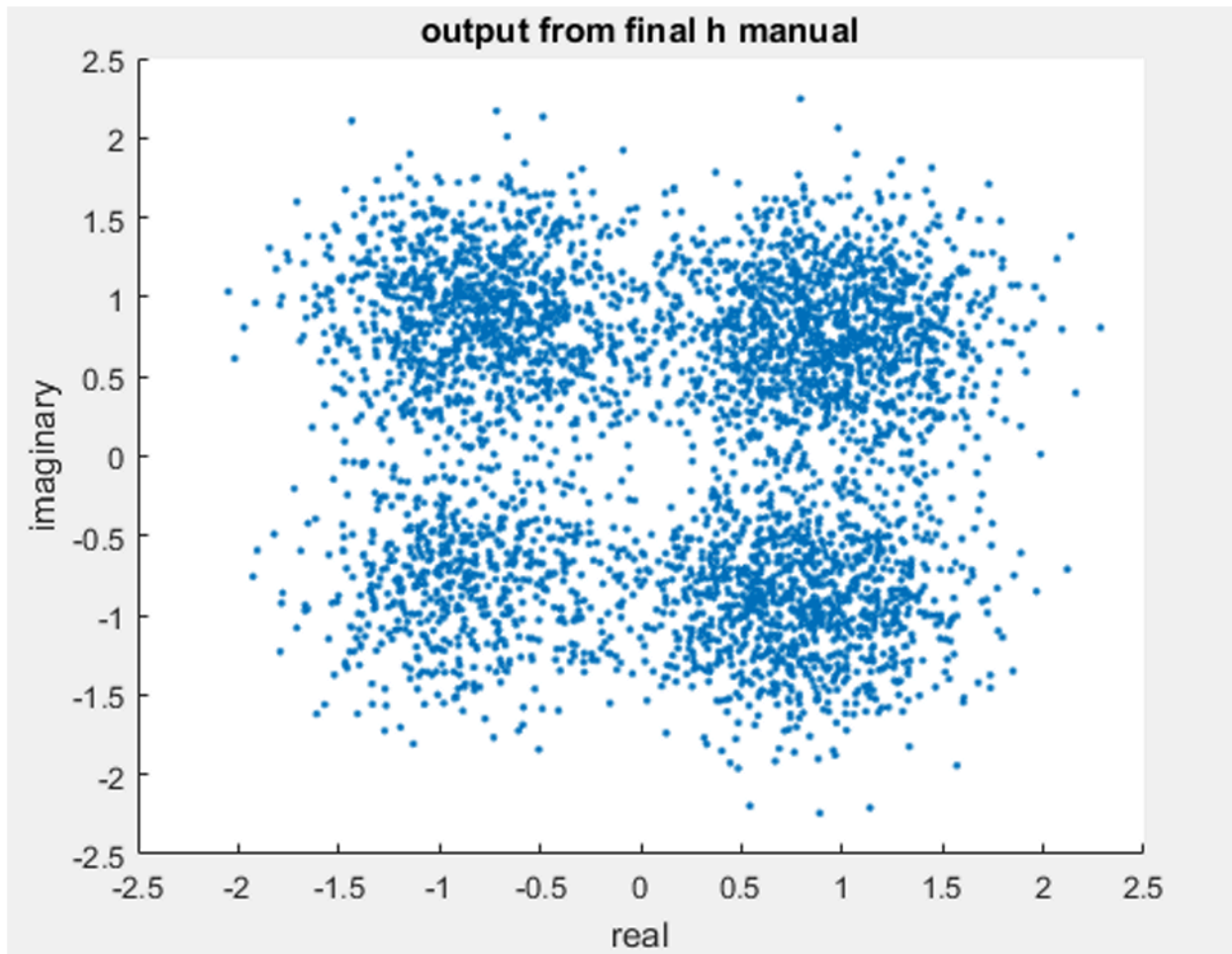


Figure 5: Constellation plot of SNR = 9db of time-invariant signals after filtering.



output from final h manually choose order 6
 Error rate is 3.575832% over 4055 QPSK symbols
 Error rate is 8.470291% over 1582 text

Time-varying dispersion-

We can achieve around 7.6% error rate give 13db SNR for time-varying signals.

We use order 6

and 10000 iterations. If it is 12db, the error rate jumps up to 14.0%. If we use order 8 and 4000

iterations, the error rate drops to 4.9%, recalling what we discussed before. The constellation plots for the "before" and "after" signals are shown below.

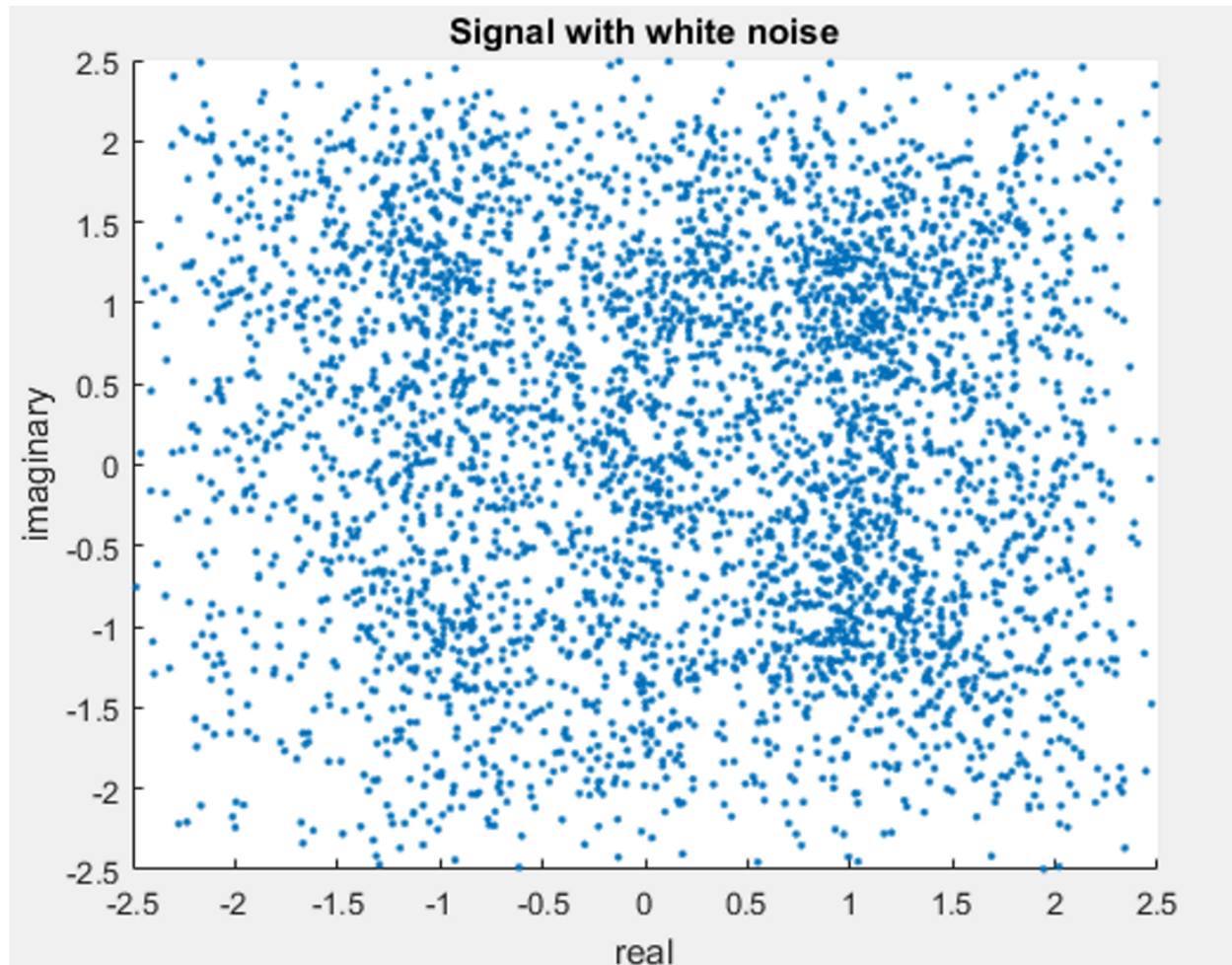


Figure 6: Constellation plot of SNR = 13db of time-varying signals before filtering

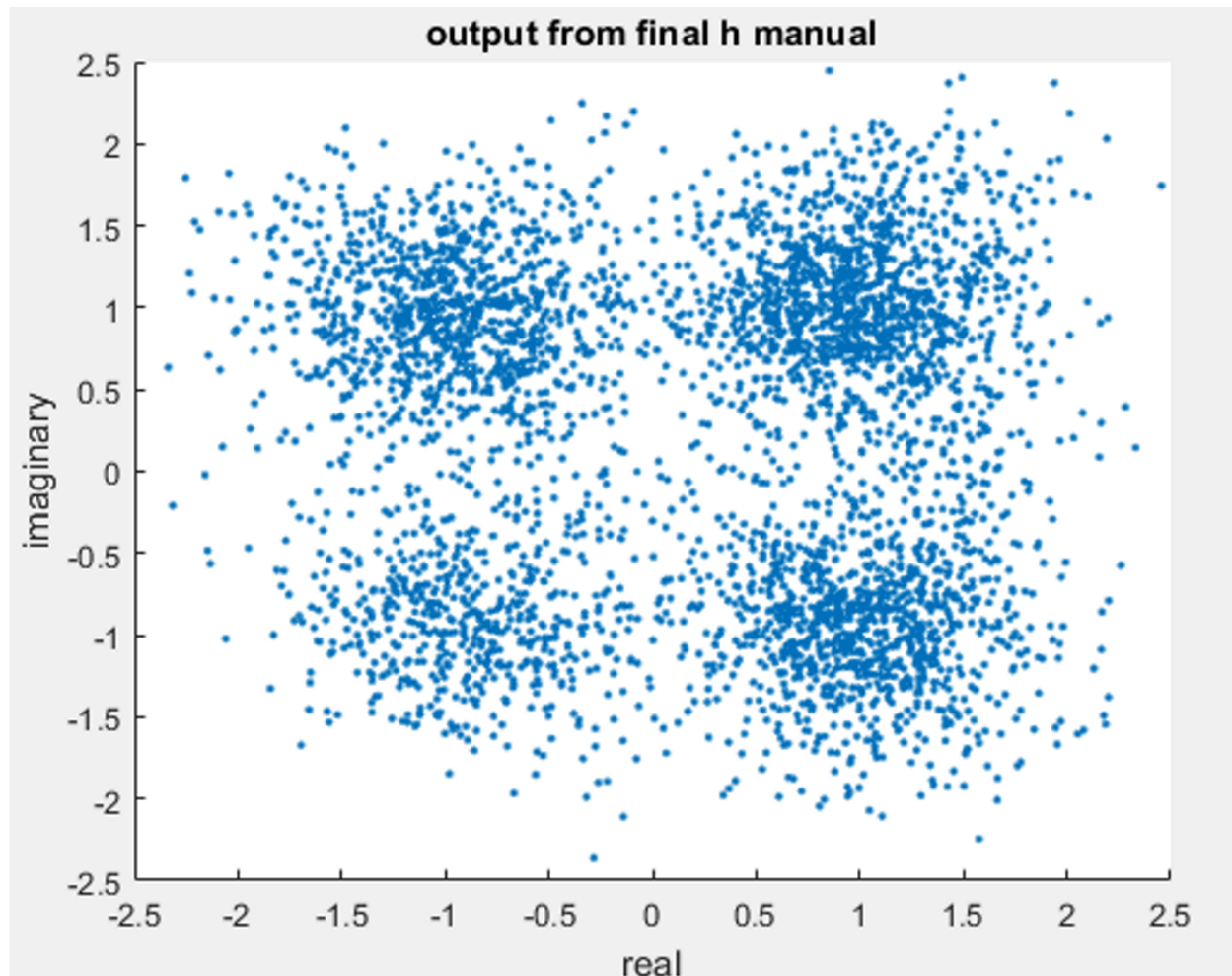


Figure 7: Constellation plot of SNR = 13db of time-varying signals after filtering



output from final h manually choose order 6
 Error rate is 3.082614% over 4055 QPSK symbols
 Error rate is 7.585335% over 1582 text

FINAL FILTER COEFFICIENT PLOT:

Noise-only-

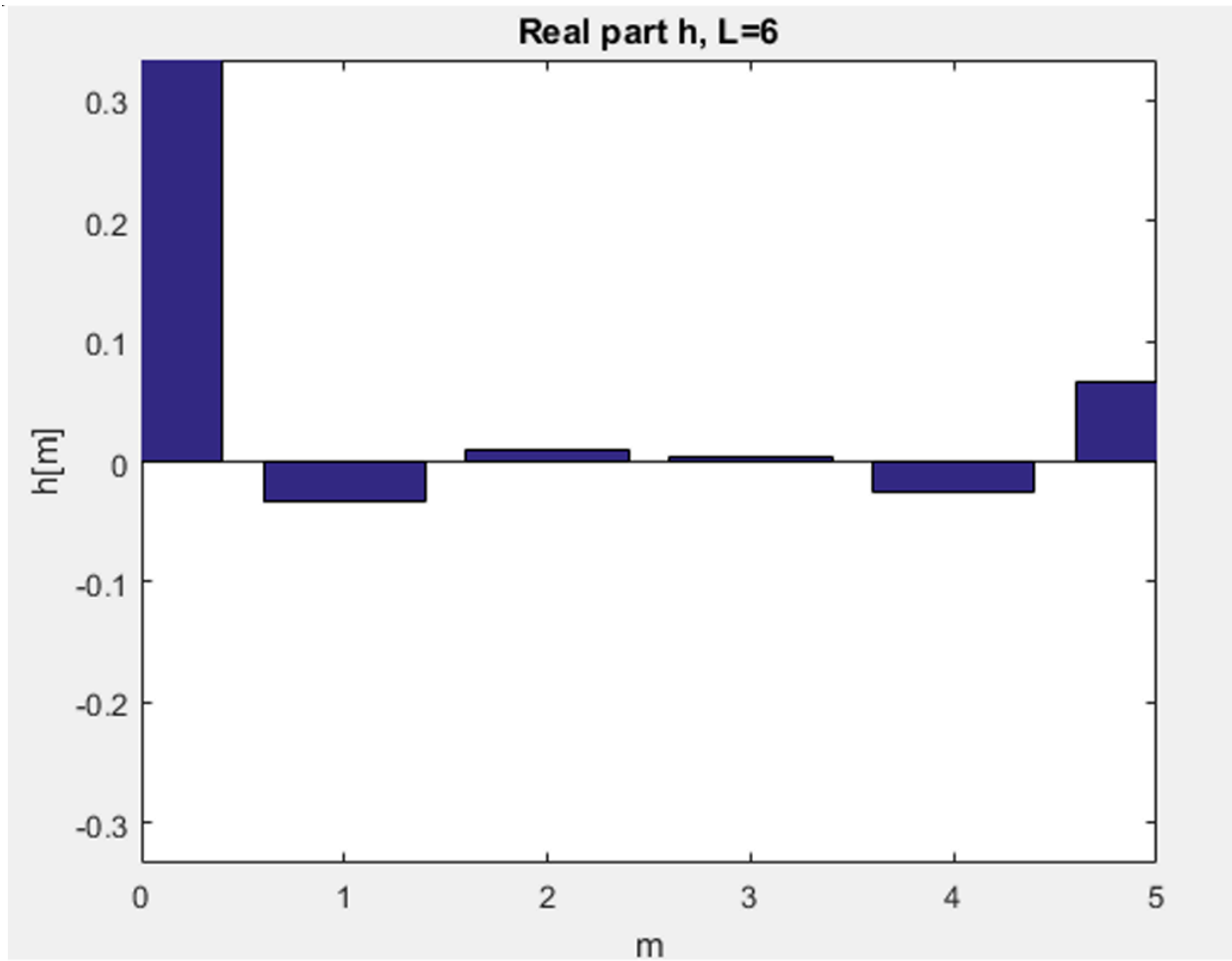


Figure 8: Final filter real coefficient of filter for 7db noise-only signals.

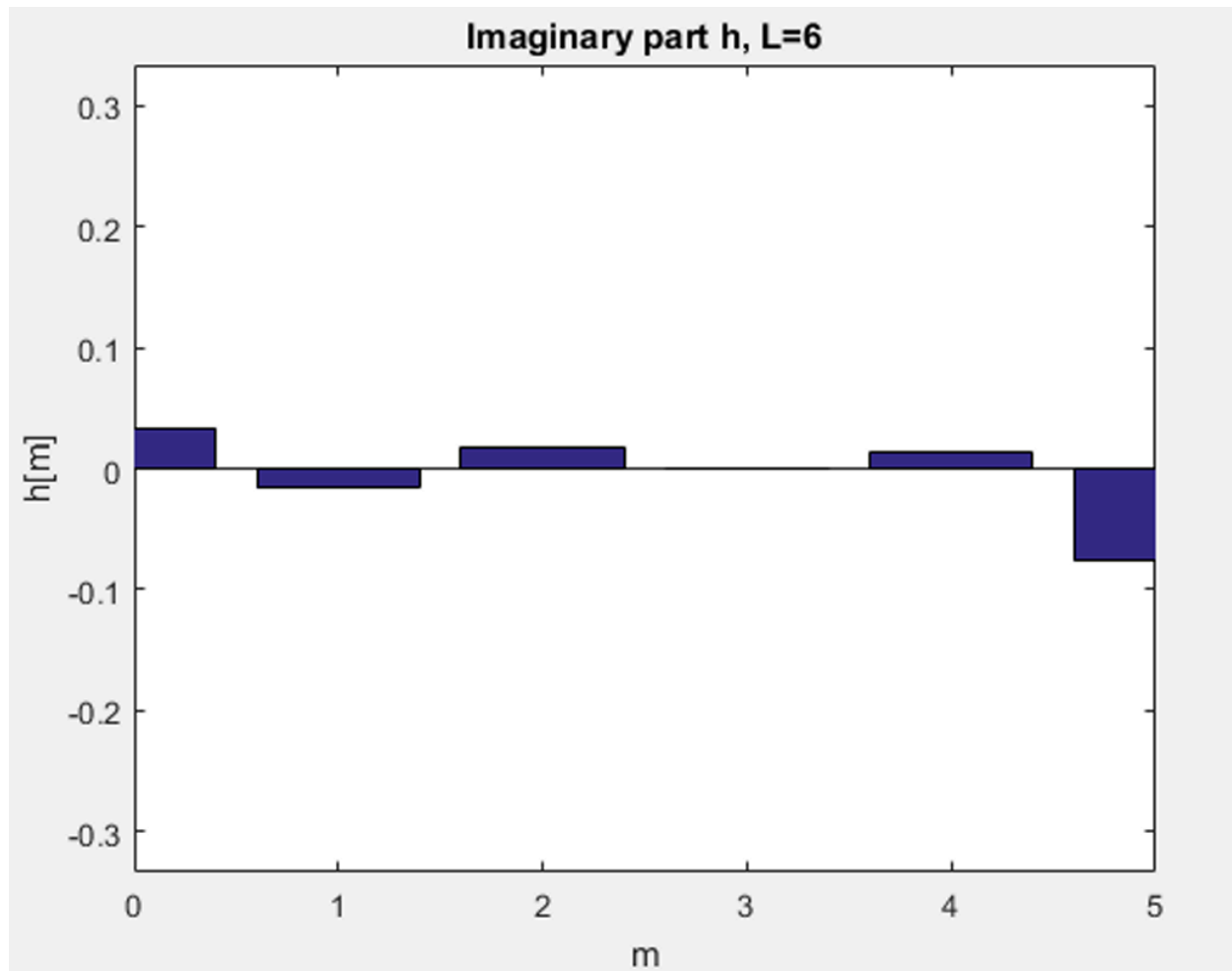


Figure 9. Final filter imaginary coefficient of filter for 7db noise-only signals.

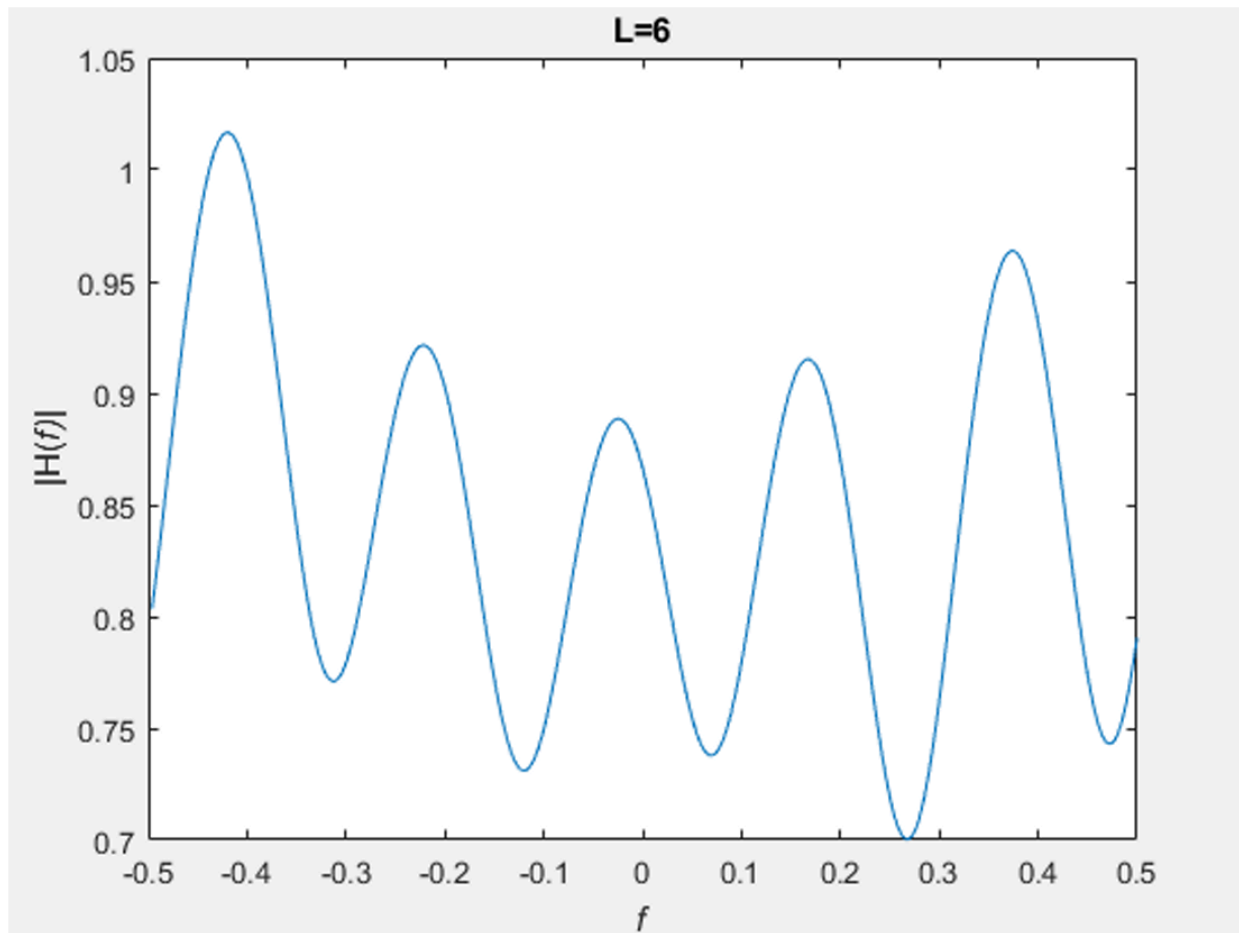


Figure 10. Final filter frequency response of filter for 7db noise-only signals.

Time-invariant-

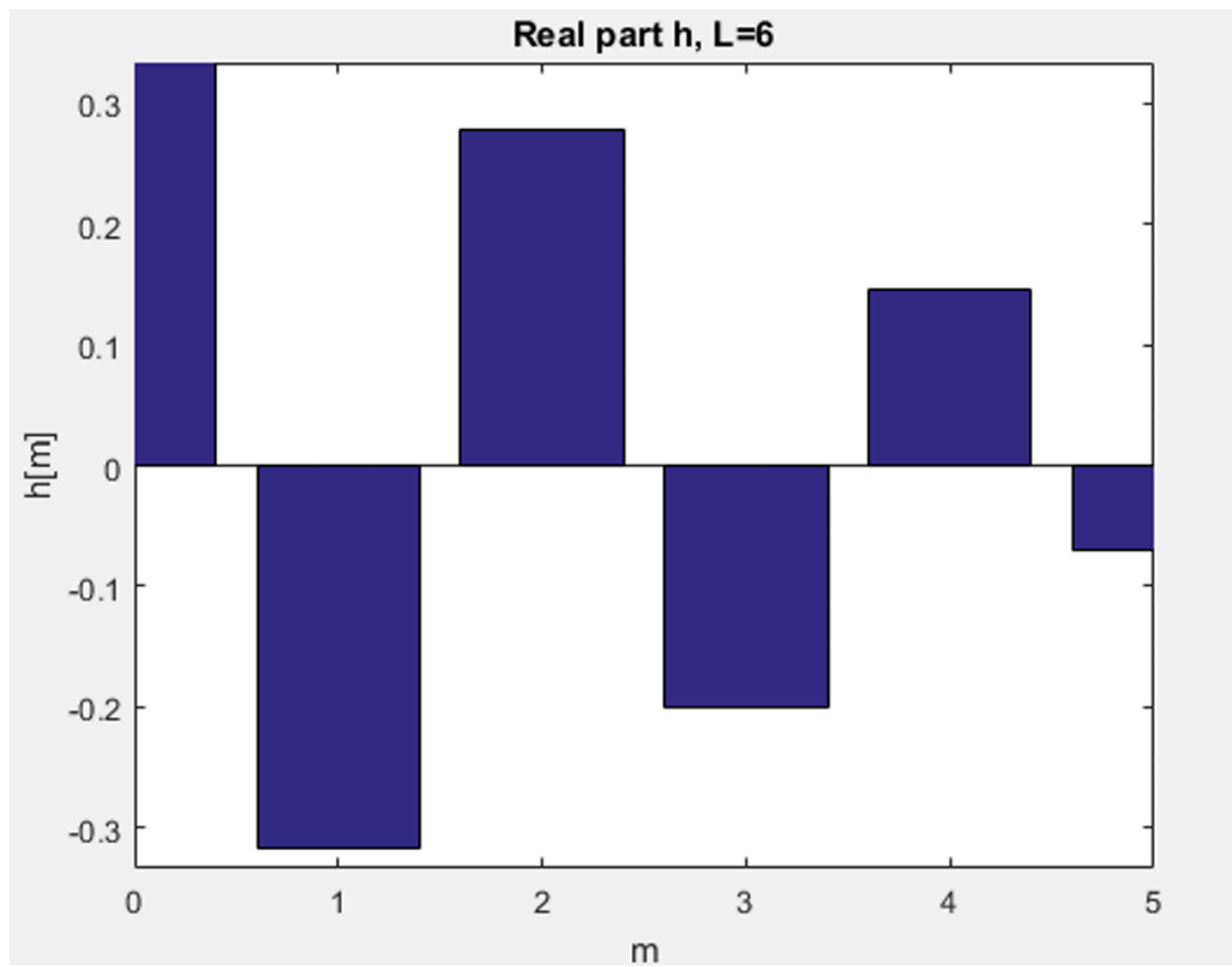


Figure 11. Final filter real coefficient of filter for 9db time-invariant signals.

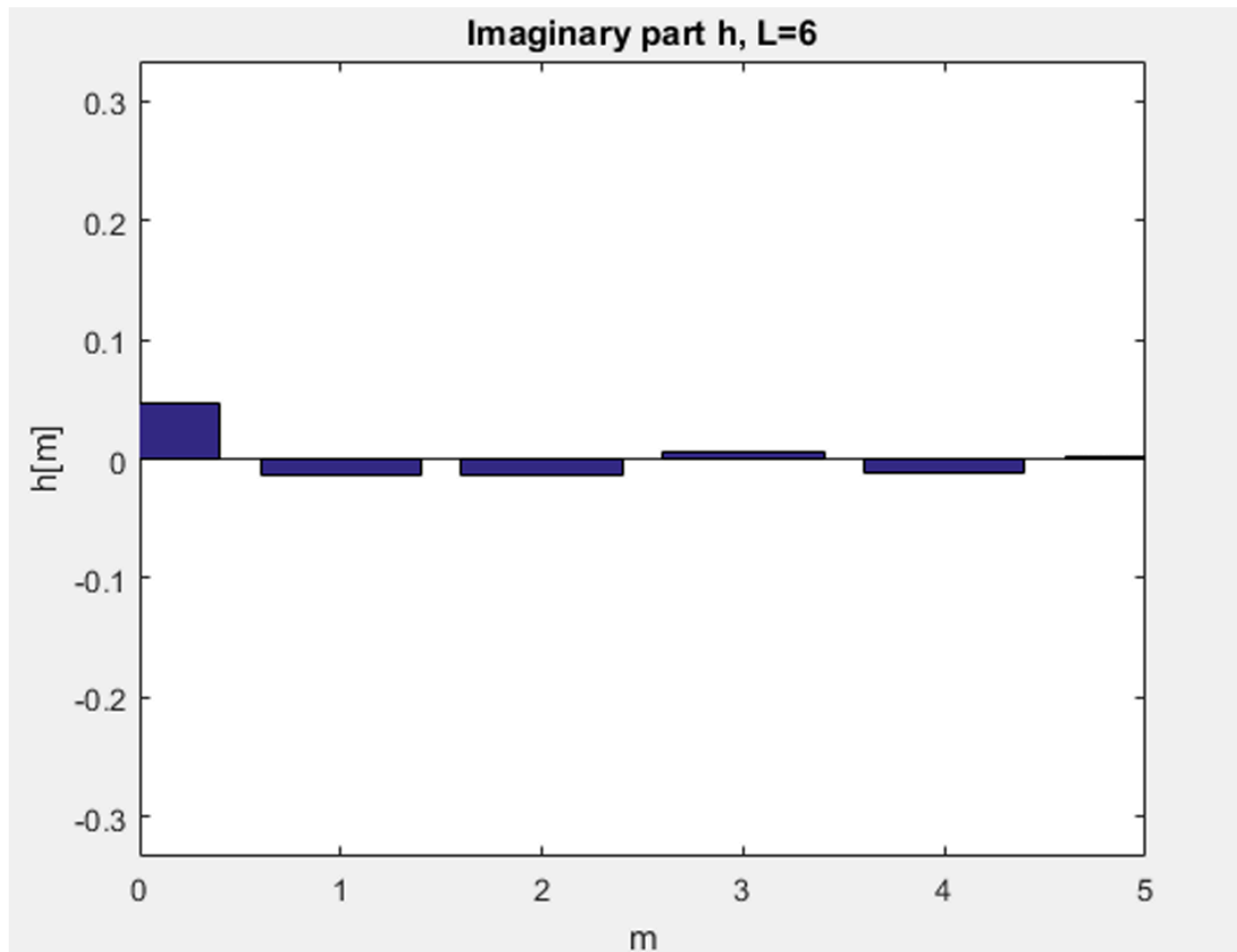


Figure 12. Final filter imaginary coefficient of filter for 9db time-invariant signals.

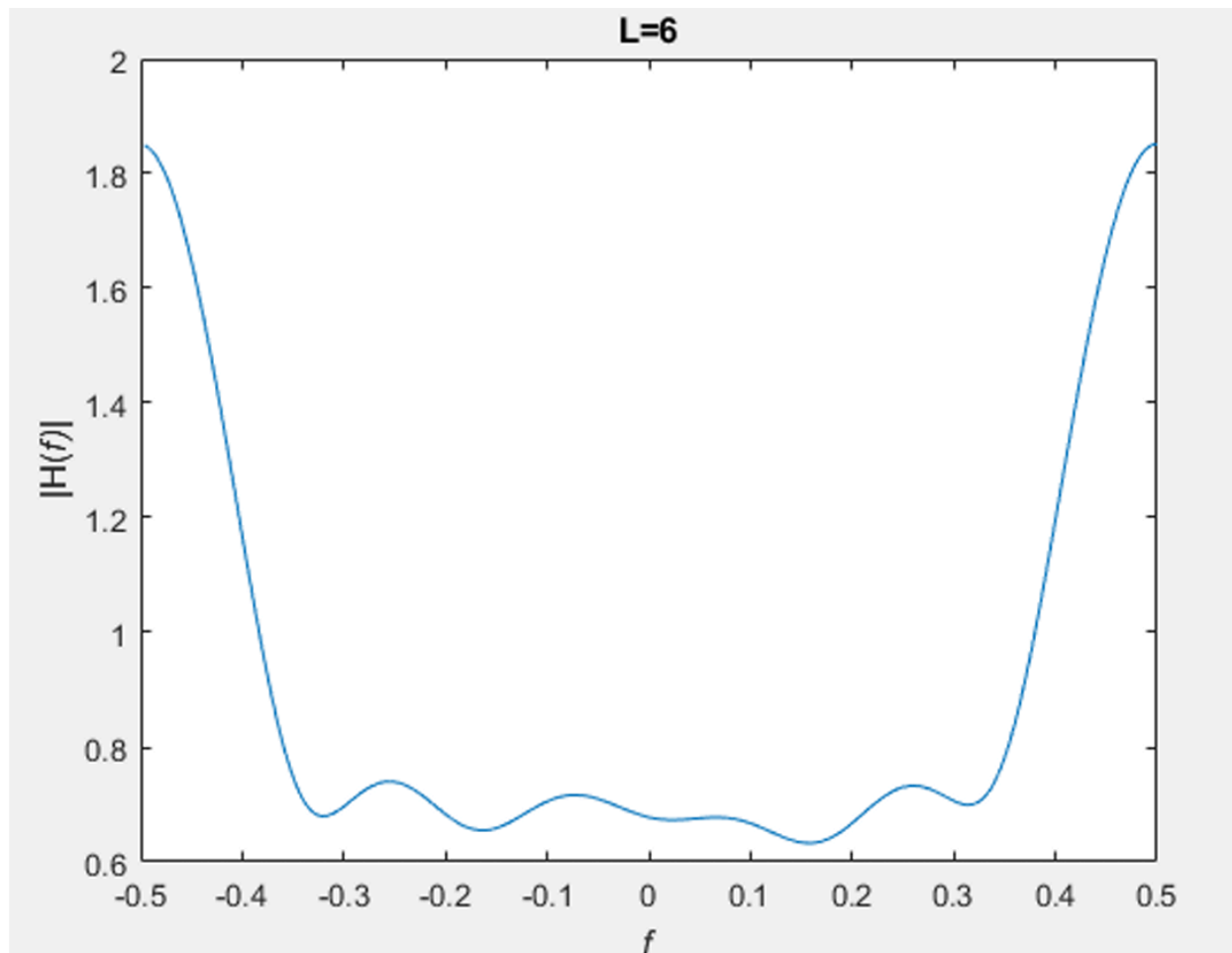


Figure 13. Final filter frequency response of filter for 9db time-invariant signals.

Time-varying Dispersion-

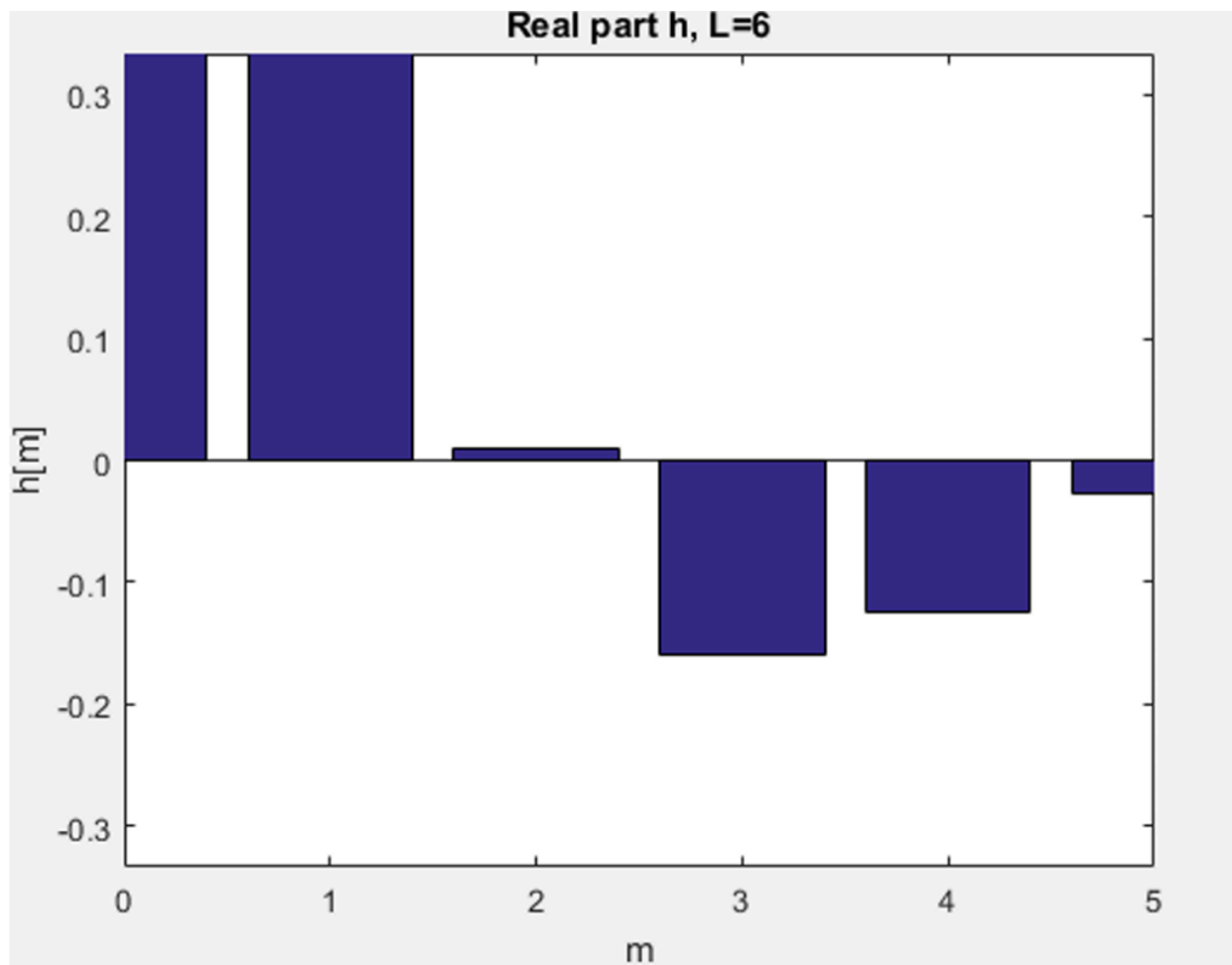


Figure 14. Final filter real coefficient of filter for 13db time-varying signals.

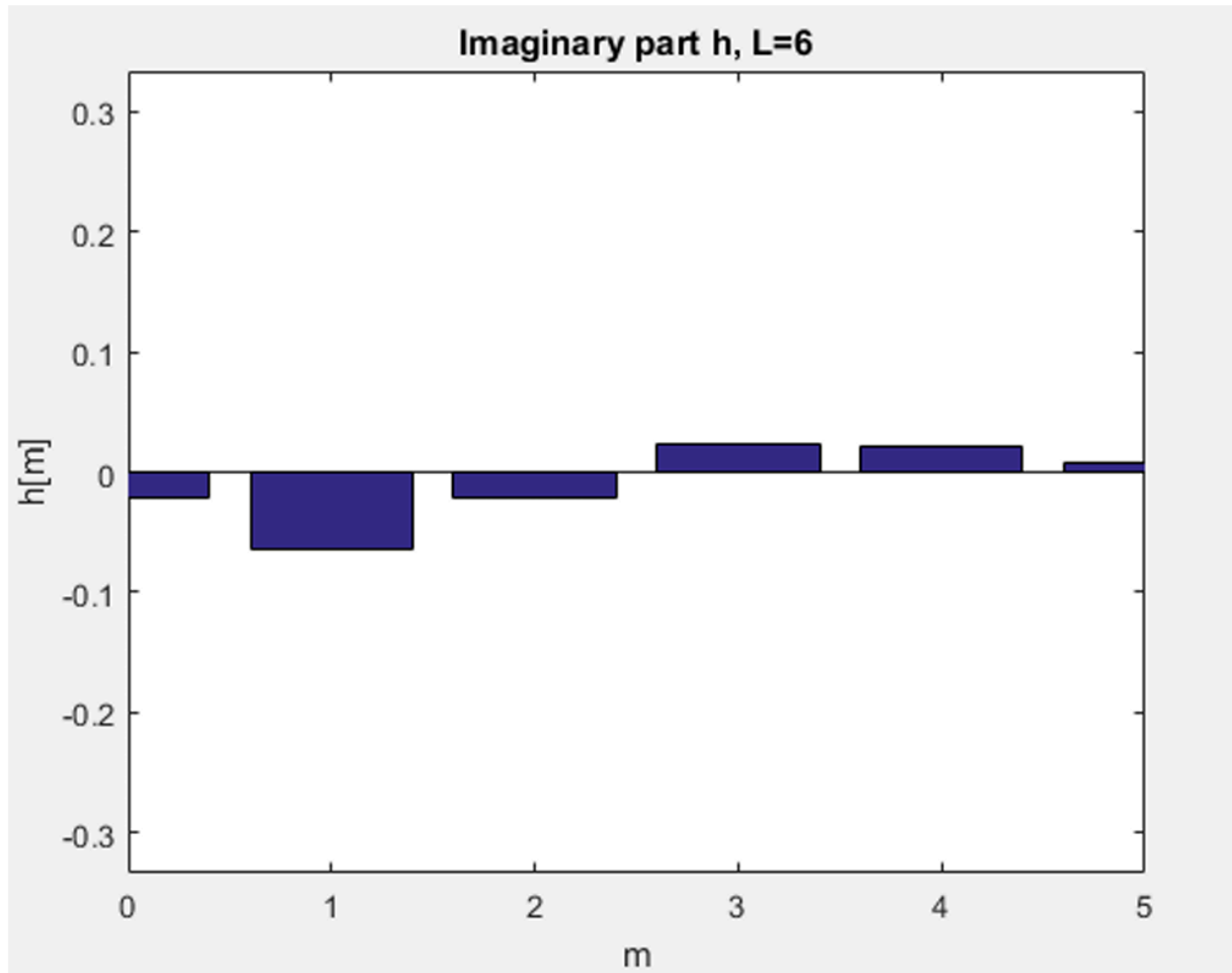


Figure 15. Final filter imaginary coefficient of filter for 13db time-varying signals.

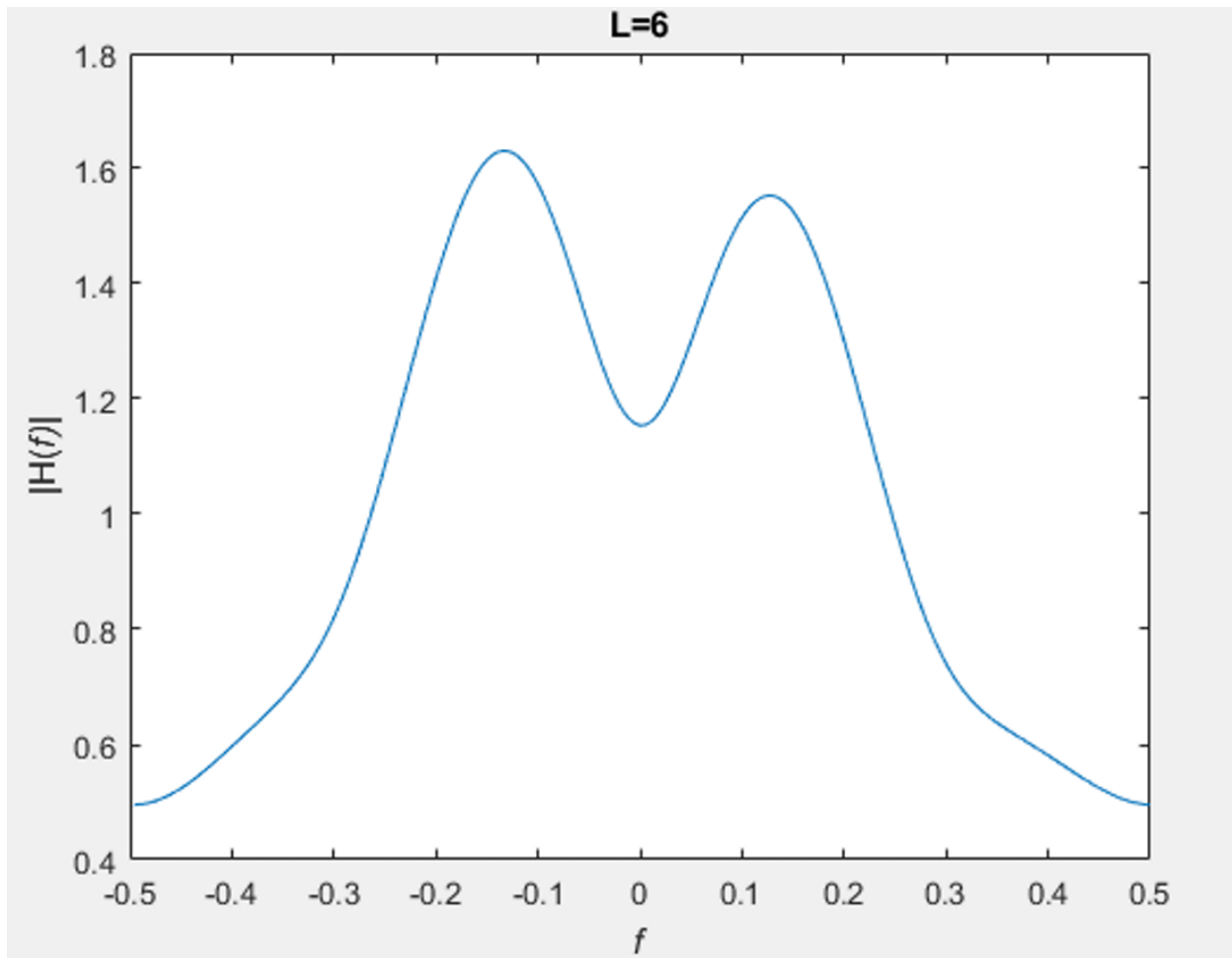


Figure 16. Final filter frequency response of filter for 13db time-varying signals.

CODE LISTING:

lms_decode(x)-

```
function message = lms_decode(x)
V = x;
load desired
% [~,~,error,snr] = lms_filter(x,2,d);
% L = 2;
% E = error;
% SNR_QPSK = snr;
% for i = 3:40;
%     [~,~,error,snr] = lms_filter(x,i,d);
%     E = [E error];
%     SNR_QPSK = [SNR_QPSK,snr];
```

```

% end
% [min_E,order_desired_E] = min(E);
% [max_SNR,order_desired_SNR] = max(SNR_QPSK);
order_desired = 6;
% [QPSK_E,h_E,~,~] = lms_filter(V,order_desired_E,d);
% [QPSK_SNR,h_SNR,~,~] = lms_filter(V,order_desired_SNR,d);
[QPSK,h,~,~] = lms_filter(V,order_desired,d);
figure
bar(0:order_desired-1,real(h)); axis([0 order_desired-1,
2/order_desired,2/order_desired]); title(sprintf('Real part h,
L=%i',order_desired))
xlabel('m'), ylabel('h[m]')
figure
bar(0:order_desired-1,imag(h)); axis([0 order_desired-1,
2/order_desired,2/order_desired]); title(sprintf('Imaginary part h,
L=%i',order_desired))
xlabel('m'), ylabel('h[m]')
figure
H = abs(fftshift(fft(h,256)));
f = [1:256]/256-0.5;
plot(f,H);title(sprintf('L=%i',order_desired))
xlabel('\itf'),ylabel('|H(\itf\it)|')
figure
scatter(real(V),imag(V),'.')
xlim([-2.5,2.5])
ylim([-2.5,2.5])
title('Signal with white noise')
xlabel('real')
ylabel('imaginary')
% figure
% scatter(real(QPSK_E),imag(QPSK_E),'.');
% title('output from final h E');
% xlabel('real')
% ylabel('imaginary')
% figure
% scatter(real(QPSK_SNR),imag(QPSK_SNR),'.');
% title('output from final h SNR');
% xlabel('real')
% ylabel('imaginary')
figure
scatter(real(QPSK),imag(QPSK),'.');
xlim([-2.5,2.5])
ylim([-2.5,2.5])
title('output from final h manual');
xlabel('real')
ylabel('imaginary')
filename = 'gettysburg.txt';
% display(sprintf('output from final h_E choose order %i',order_desired_E))
% [txt_test_E, error_rate_QPSK_E, error_rate_txt_E] =
evaluation(filename,QPSK_E,order_desired_E);
% display(sprintf('output from final h_SNR choose
order %i',order_desired_SNR))

```

```
% [txt_test_SNR, error_rate_QPSK_SNR, error_rate_txt_SNR] =
evaluation(filename,QPSK_SNR,order_desired_SNR);
display(sprintf('output from final h manually choose
order %i',order_desired))
[txt_test, error_rate, error_rate] = evaluation(filename,QPSK,order_desired);
message = txt_test;
end
```

This is the **main decoding function** of the project. It takes a noisy QPSK signal as input, applies the LMS adaptive filter, visualizes the results, evaluates the recovered message, and finally returns the decoded text.

Overall Workflow

```
Noisy Signal
  ↓
  LMS Filter
  ↓
Filtered QPSK Symbols
  ↓
Visualization
  ↓
Evaluation
  ↓
Decoded Text Message
```

Important Code Sections

Store Input Signal

```
V = x;
```

Stores the received noisy signal in `V`.

Load Desired Signal

```
load desired
```

Loads the known reference signal (`d`) used for LMS training.

This desired signal helps the adaptive filter learn the correct channel characteristics.

Filter Order Selection

```
order_desired = 6;
```

Selects a filter order of 6 taps.

The commented code above shows that different filter orders (2–40) were tested, but order 6 was chosen manually based on performance.

This is one of the most important parameters in the system.

Train LMS Filter

```
[QPSK,h,~,~] = lms_filter(V,order_desired,d);
```

Most Important Line

This calls the LMS adaptive filter.

Outputs:

- `QPSK` → Filtered QPSK symbols
- `h` → Final learned filter coefficients

At this point, the noisy signal has been processed and cleaned.

Visualize Filter Coefficients

```
bar(0:order_desired-1,real(h));
```

Displays the real part of the learned filter coefficients.

```
bar(0:order_desired-1,imag(h));
```

Displays the imaginary part of the learned filter coefficients.

These plots help us understand how the adaptive filter has adjusted itself during training.

Frequency Response of Filter

```
H = abs(fftshift(fft(h,256)));
```

Computes the frequency response of the trained filter.

```
plot(f,H);
```

Plots the filter response in the frequency domain.

This shows which frequencies are being emphasized or suppressed by the LMS filter.

Visualize Noisy Signal

```
scatter(real(V),imag(V),'.')
```

Plots the constellation diagram of the received noisy signal.

A noisy constellation appears scattered and difficult to decode.

Visualize Filtered Signal

```
scatter(real(QPSK),imag(QPSK),'.');
```

★ Very Important

Plots the constellation after LMS filtering.

Ideally, the points should cluster around the four QPSK symbol locations, indicating successful noise reduction.

Select Ground Truth File

```
filename = 'gettysburg.txt';
```

Specifies the original text file used for evaluation.

This serves as the ground truth message.

Evaluate Decoding Performance

```
[txt_test, error_rate, error_rate] = evaluation(filename,QPSK,order_desired);
```

★ Most Important Line

This function:

1. Converts filtered QPSK symbols back to text.
2. Compares the recovered message with the original text.
3. Calculates decoding error rates.

Return Decoded Message

```
message = txt_test;
```

Stores the recovered text message as the function output.

Commented Sections

Several blocks are commented out:

```
% for i = 3:40
```

These were used to automatically test different filter orders and find:

```
% minimum error
```

```
% maximum SNR
```

The final implementation instead uses:

```
order_desired = 6;
```

because it produced good overall performance.

Summary

`lms_decode.m` is the top-level controller of the entire project. It loads the reference signal, trains an LMS adaptive filter, applies the filter to the noisy QPSK signal, visualizes the filter performance using constellation and frequency-response plots, evaluates the recovered text against the original message, and returns the final decoded text. The most important step is the call to `lms_filter()`, which performs the actual adaptive noise reduction and signal recovery.

lms_filter(V,L,d)-

```
function[QPSK,h,MSE,SNR_QPSK] = lms_filter(V,L,d)
N = length(V);
n = randn(length(V),1)+j*randn(length(V),1); n = n/sqrt(var(n));
SNRdB = snr(V,n);
SNR = 10^(SNRdB/10);
S = sqrt(2*SNR);
mu_max = 1/L/(S^2/2);
mu = mu_max/100;
h = zeros(L,1);
% h = 2*(rand(L,1)-0.5)*(2/L);
% E = [];
x_arr = []; % d is desired signal
d_arr = [];
x = V(1:length(d));
for i = 1:ceil(N/L)
x_arr = [x_arr;x];
d_arr = [d_arr;d];
end
MSE_arr = [];
iterations = 1000:1000:10000;
% iterations = length(V);
for N = iterations
for k = L:N
xk = x_arr(k:-1:k-L+1);
output = (h')*xk;
error = d_arr(k) - output;
```



```

%      mu_k = 0.5/(xk'xk+0.001);
%      h = h + mu_kconj(error)xk;
h = h+muconj(error) (xk);
end
self_test = conv(h',x);
E = d - self_test(L:end);
MSE = sum(abs(E).^2)/N;
MSE_arr = [MSE_arr MSE];
end
% plot(iterations,MSE_arr)
% title('MSE vs iterations')
% xlabel('iterations')
% ylabel('MSE')
QPSK = conv(h',V);
QPSK = QPSK(L:length(V));
SNR_QPSK = snr(QPSK,n(L:length(V)));
end

```

This file contains the **core LMS adaptive filtering algorithm**. Its job is to train a filter using a known reference signal and then use the learned filter coefficients to remove noise and recover the transmitted QPSK symbols.

Inputs and Outputs

Inputs

V

Received noisy QPSK signal.

L

Filter order (number of filter taps).

d

Desired/reference signal used for training.

Outputs

QPSK

Filtered QPSK symbols.

h

Final trained filter coefficients.

MSE

Mean Squared Error after training.

SNR_QPSK

Output SNR after filtering.

Important Code Sections

Signal Length

```
N = length(V);
```

Stores the length of the received signal.

Generate Reference Noise

```
n = randn(length(V),1)+j*randn(length(V),1);  
n = n/sqrt(var(n));
```

Generates complex Gaussian noise with unit variance.

This noise is later used for SNR calculations.

Estimate Signal Power and SNR

```
SNRdB = snr(V,n);  
SNR = 10^(SNRdB/10);  
S = sqrt(2*SNR);
```

Computes the signal-to-noise ratio and estimates the signal amplitude.

These values are used to determine a stable LMS step size.

Step Size Selection

```
mu_max = 1/L/(S^2/2);  
mu = mu_max/100;
```

Computes the maximum stable LMS step size and then selects a much smaller value for stability.

A smaller step size results in slower but more reliable convergence.

Filter Initialization

```
h = zeros(L,1);
```

Initializes all filter coefficients to zero.

We found that zero initialization provided stable performance and slightly better accuracy than random initialization.

Create Training Sequences

```
x_arr = [];  
d_arr = [];
```

Creates arrays that will hold the training data.

```
x = V(1:length(d));
```

Extracts the portion of the received signal corresponding to the desired signal length.

```
for i = 1:ceil(N/L)
    x_arr = [x_arr;x];
    d_arr = [d_arr;d];
end
```

Repeats the noisy signal and desired signal until enough samples are available for LMS training.

This ensures that the training data is long enough for all iterations.

Iteration Experiment

```
iterations = 1000:1000:10000;
```

Tests LMS performance for different numbers of training iterations.

Used to study how convergence changes as the number of updates increases.

LMS Adaptation Loop

```
for k = L:N
```

Begins the LMS training process.

The loop starts at **L** to ensure enough previous samples are available to form the filter input vector.

Form Input Vector

```
xk = x_arr(k:-1:k-L+1);
```

Collects the most recent **L** samples and forms the input vector for the filter.

Compute Filter Output

```
output = (h')*xk;
```

Calculates the filter output using the current filter coefficients.

Compute Error

```
error = d_arr(k) - output;
```

Measures the difference between the desired signal and the filter output.

This error drives the learning process.

LMS Coefficient Update

```
h = h + mu*conj(error)*(xk);
```

★ Most Important Line in the Entire Project

Updates the filter coefficients according to the LMS adaptation rule.

The filter gradually adjusts itself to minimize the error and learn the channel characteristics.

Compute MSE

```
self_test = conv(h',x);
```

Applies the trained filter to the training signal.

```
E = d - self_test(L:end);
```

Computes the remaining error after training.

```
MSE = sum(abs(E).^2)/N;
```

Calculates the Mean Squared Error.

Lower MSE indicates better filter performance.

Apply Final Filter

```
QPSK = conv(h',V);
```

Applies the trained filter to the entire noisy signal.

This produces the recovered QPSK symbols.

```
QPSK = QPSK(L:length(V));
```

Removes the initial filter delay introduced by convolution.

This aligns the filtered signal correctly.

Compute Output SNR

```
SNR_QPSK = snr(QPSK,n(L:length(V)));
```

Calculates the SNR after filtering.

This provides a measure of how much the LMS filter improved signal quality.

Summary

`lms_filter.m` is the heart of the project. It trains an LMS adaptive filter using a known reference signal, continuously updates the filter coefficients to minimize error, and then applies the learned filter to the noisy received signal. The file also evaluates performance using MSE and SNR metrics. The most critical operation is the LMS coefficient update equation, which enables the filter to adapt and recover cleaner QPSK symbols from the noisy channel.

genSignal.m-

```
load gettysburg  
load moreSignals  
x10=genChannel(s,0,1,10);
```

```

x10_ti=genChannel(s,1,1,10);
x10_tv=genChannel(s,2,1,10);
x6=genChannel(s,0,1,6);
x6_ti=genChannel(s,1,1,6);
x6_tv=genChannel(s,2,1,6);
x7=genChannel(s,0,1,7);
x7_ti=genChannel(s,1,1,7);
x7_tv=genChannel(s,2,1,7);
x8=genChannel(s,0,1,8);
x8_ti=genChannel(s,1,1,8);
x8_tv=genChannel(s,2,1,8);
x9=genChannel(s,0,1,9);
x9_ti=genChannel(s,1,1,9);
x9_tv=genChannel(s,2,1,9);
x11=genChannel(s,0,1,11);
x11_ti=genChannel(s,1,1,11);
x11_tv=genChannel(s,2,1,11);
x12=genChannel(s,0,1,12);
x12_ti=genChannel(s,1,1,12);
x12_tv=genChannel(s,2,1,12);
x13=genChannel(s,0,1,13);
x13_ti=genChannel(s,1,1,13);
x13_tv=genChannel(s,2,1,13);
x5=genChannel(s,0,1,5);
x5_ti=genChannel(s,1,1,5);
x5_tv=genChannel(s,2,1,5);
x0=genChannel(s,0,1,0);
x0_ti=genChannel(s,1,1,0);
x0_tv=genChannel(s,2,1,0);
save('AllSignals','x0','x0_ti','x0_tv','x5','x5_ti','x5_tv','x6','x6_ti','x6_
tv','x7','x7_ti','x7_tv','x8','x8_ti','x8_tv','x9','x9_ti','x9_tv','x10','x10_
_ti','x10_tv','x12','x12_ti','x12_tv','x11','x11_ti','x11_tv','x13','x13_ti',
'x13_tv','x15','x15_ti','x15_tv','x20','x20_ti','x20_tv')

```

This file is used to **generate test signals** under different channel conditions and SNR levels. These generated signals are later used to evaluate the performance of the LMS-based QPSK decoder.

Instead of testing the decoder on a single noisy signal, we create multiple versions of the transmitted signal with varying amounts of noise and different channel behaviors.

Overall Functionality

Original QPSK Signal

↓

genChannel()

↓

```
Different SNR Levels
↓
Time-Invariant & Time-Varying Channels
↓
AllSignals.mat
↓
Used for LMS Testing
```

Important Code Sections

Load Input Data

```
load gettysburg
load moreSignals
```

Loads the original transmitted signal and supporting signal data needed to generate channel outputs.

Generate Signals at Different SNR Levels

```
x10 = genChannel(s,0,1,10);
```

Generates a noisy signal with **10 dB SNR**.

Parameters:

- `s` → Original transmitted signal
- `0` → Channel type
- `10` → SNR value

Generate Time-Invariant Signals

```
x10_ti = genChannel(s,1,1,10);
```

Generates a signal that passes through a **time-invariant channel**.

In a time-invariant channel, the channel characteristics remain constant throughout transmission.

Generate Time-Varying Signals

```
x10_tv = genChannel(s,2,1,10);
```

Generates a signal that passes through a **time-varying channel**.

In this case, the channel characteristics change over time, making signal recovery more challenging.

Multiple SNR Levels

The same process is repeated for several SNR values:

```
x0  
x5  
x6  
x7  
x8  
x9  
x10  
x11  
x12  
x13
```

These signals allow us to test decoder performance under different noise conditions.

Lower SNR:

- More noise
- Harder decoding

Higher SNR:

- Less noise
- Easier decoding

Save All Generated Signals

```
save('AllSignals', ...)
```

★ Most Important Line

Stores all generated signals into a single MATLAB file called `AllSignals.mat`.

This file is later loaded by:

```
load AllSignals
```

inside `main.m`.

Without this step, the decoder would have no test signals to process.

Signal Naming Convention

Signal	Description
x10	10 dB noisy signal
x10_ti	10 dB time-invariant channel
x10_tv	10 dB time-varying channel
x5	5 dB noisy signal
x5_ti	5 dB time-invariant channel
x5_tv	5 dB time-varying channel

and similarly for other SNR values.

Summary

`genSignal.m` generates a collection of noisy QPSK signals under different SNR levels and channel conditions. For each SNR value, three versions of the signal are created: a standard noisy signal, a time-invariant channel signal, and a time-varying channel signal. All generated signals are then saved into `AllSignals.mat`, which serves as the dataset used to test and evaluate the LMS decoder under various communication scenarios.

evaluation(filename,QPSK_test,order_desired)-

```
function [txt_test, error_rate_QPSK,  
error_rate_txt]=evaluation(filename,QPSK_test,order_desired)
```

```

head_signal = 85; %[0:31 4 5 asciiseq(find(~isnan(asciiseq))) 0 0 0 0 0 0],
[0:31 4 5] length is 34, 34 5/2 = 85
[QPSK_real, ascii] = file2bin(filename);
txt_real = char(ascii);
txt_test = bin2text(QPSK_test(head_signal+1-order_desired+1:end));
txt_test=txt_test(1:length(txt_real));
dot_idx = find(filename == '.');
outfilename = strcat(filename(1:dot_idx-1),'_out',filename(dot_idx:end));
fileID_out = fopen(outfilename,'w');
fwrite(fileID_out,txt_test,'uint8');
fclose(fileID_out);
[QPSK_test, ~] = file2bin(outfilename);
error_rate_QPSK = length(find(QPSK_real ~= QPSK_test))/length(QPSK_real);
error_rate_txt = length(find(txt_real ~= txt_test))/length(txt_real);
display(sprintf('Error rate is %f%% over %d QPSK
symbols',error_rate_QPSK 100,length(QPSK_real)));
display(sprintf('Error rate is %f%% over %d
text',error_rate_txt*100,length(txt_real)));
end

```

This file evaluates the performance of the LMS decoder by comparing the recovered message with the original message stored in `gettysburg.txt`. It calculates both the **QPSK symbol error rate** and the **text character error rate**.

Overall Functionality

```

Original Text File
    ↓
  file2bin()
    ↓
Ground Truth QPSK Symbols
    ↓
Compare with Decoded Output
    ↓
Calculate Error Rates
    ↓
Display Performance

```

Important Code Sections

Function Definition

```

function [txt_test, error_rate_QPSK, error_rate_txt] = evaluation(filename,QPSK_test,orde
r_desired)

```

Takes:

- `filename` → Original text file
- `QPSK_test` → Decoded QPSK symbols
- `order_desired` → LMS filter order

Returns:

- Recovered text
- QPSK symbol error rate
- Text character error rate

Synchronization Offset

```
head_signal = 85;
```

The transmitted signal begins with a synchronization/header sequence before the actual message.

We skip the first 85 QPSK symbols to reach the start of the real message.

This value comes from:

```
[0:31 4 5]
```

which contains 34 symbols.

Since each symbol is represented using 5 bits and QPSK carries 2 bits per symbol:

```
34 × 5 / 2 = 85 QPSK symbols
```

Generate Ground Truth

```
[QPSK_real, ascii] = file2bin(filename);
```

Converts the original text file into:

- Reference QPSK symbols (`QPSK_real`)
- Reference ASCII values (`ascii`)

These serve as the correct answer for comparison.

Original Text

```
txt_real = char(ascii);
```

Converts the ASCII values into readable text.

This is the original message that was transmitted.

Recover Decoded Text

```
txt_test = bin2text(QPSK_test(head_signal+1-order_desired+1:end));
```

★ Most Important Line

Converts the filtered QPSK symbols back into text.

Before conversion, we:

- Skip the synchronization sequence (`head_signal`)
- Compensate for filter delay (`order_desired - 1`)

This ensures that the decoded message is properly aligned with the original message.

Match Text Length

```
txt_test = txt_test(1:length(txt_real));
```

Trims the decoded text so that it has the same length as the original message.

This allows a fair comparison.

Save Decoded Text

```
outfilename = strcat(filename(1:dot_idx-1), '_out', filename(dot_idx:end));
```

Creates an output filename such as:

```
gettysburg_out.txt
```

```
fwrite(fileID_out,txt_test,'uint8');
```

Writes the recovered text into the output file.

This allows the decoded message to be inspected manually.

Convert Output Back to QPSK

```
[QPSK_test, ~] = file2bin(outfilename);
```

Converts the recovered text back into QPSK symbols.

This makes it possible to compare symbol-by-symbol against the original transmission.

QPSK Symbol Error Rate

```
error_rate_QPSK = length(find(QPSK_real ~= QPSK_test)) / length(QPSK_real);
```

Computes the percentage of QPSK symbols that were decoded incorrectly.

Lower values indicate better symbol recovery.

Text Error Rate

```
error_rate_txt = length(find(txt_real ~= txt_test)) / length(txt_real);
```

Computes the percentage of characters that differ from the original message.

This directly measures message recovery accuracy.

Display Results

```
display(sprintf('Error rate is %f%% over %d QPSK symbols',error_rate_QPSK*100,length(QPSK_real)));
```

Displays the QPSK symbol error rate.

```
display(sprintf('Error rate is %f%% over %d text',error_rate_txt*100,length(txt_real)));
```

Displays the text character error rate.

Summary

`evaluation.m` measures how well the LMS decoder performs by comparing the recovered message with the original text file. It removes the synchronization sequence and filter delay, converts the decoded QPSK symbols back into text, and calculates both symbol-level and character-level error rates. These metrics provide a quantitative measure of the decoder's accuracy and overall performance.

bin2text(bs)-

```
function text = bin2text(bs)
bn_length = 5;
Threshold = 0;
bits = [];
for index = 1:length(bs)
b1 = real(bs(index));
b2 = imag(bs(index));

if(b1 < -Threshold)
b1 = '1';
elseif(b1 > Threshold)
b1 = '0';
else
display('threshold situation detected')
b1 = '1';
end
if(b2 < -Threshold)
b2 = '1';
elseif(b2 > Threshold)
b2 = '0';
```

```

else
b2 = '1';
display('threshold situation detected')
end
bits = [bits b1 b2];
end
demodBn = reshape(bits, bn_length , length(bits)/bn_length);
demodBn = demodBn';
ascii = bin2dec(demodBn);
ascii = ascii';
ascii(ascii >= 6 & ascii <= 31) = ascii(ascii >= 6 & ascii <= 31) + 91;% bn
to a-z

ascii(ascii == 0) = 32;% bn to ' '
ascii(ascii == 1) = 44;% bn to ','
ascii(ascii == 2) = 46;% bn to '.'
ascii(ascii == 3) = 39;% bn to '''
ascii(ascii == 4) = 10;% bn to line feed
ascii(ascii == 5) = 13;% bn to carriage return
text = char(ascii);
end

```

This file performs **QPSK demodulation and text reconstruction**. It converts the filtered QPSK symbols back into binary bits, maps those bits to the custom ASCII encoding used in the project, and finally reconstructs the original text message.

Overall Functionality

```

Filtered QPSK Symbols
    ↓
Extract Real & Imaginary Parts
    ↓
Convert Symbols to Bits
    ↓
Group Bits into 5-bit Codes
    ↓
Convert to ASCII Values
    ↓
Recover Original Text

```

Important Code Sections

Define Encoding Parameters

```
bn_length = 5;
```



```
Threshold = 0;
```

- `bn_length = 5` means each character is represented using 5 bits.
- `Threshold = 0` is used to decide whether a QPSK component represents a binary 0 or 1.

Process Each QPSK Symbol

```
for index = 1:length(bs)
```

Loops through every received QPSK symbol.

Extract Real and Imaginary Components

```
b1 = real(bs(index));  
b2 = imag(bs(index));
```

Each QPSK symbol carries 2 bits:

- Real part → First bit
- Imaginary part → Second bit

Convert Real Part to Bit

```
if(b1 < -Threshold)  
    b1 = '1';  
elseif(b1 > Threshold)  
    b1 = '0';  
else  
    b1 = '1';  
end
```

Uses threshold detection to determine whether the first bit is 0 or 1.

Convert Imaginary Part to Bit

```

if(b2 < -Threshold)
    b2 = '1';
elseif(b2 > Threshold)
    b2 = '0';
else
    b2 = '1';
end

```

Performs the same decision process for the second bit.

Build Bit Stream

```
bits = [bits b1 b2];
```

★ Important Line

Appends the recovered bits to the complete binary sequence.

After all symbols are processed, this array contains the entire decoded bit stream.

Group Bits into Characters

```

demodBn = reshape(bits, bn_length, length(bits)/bn_length);
demodBn = demodBn';

```

Groups the recovered bits into 5-bit blocks.

Each block corresponds to one encoded character.

Convert Binary to Decimal

```
ascii = bin2dec(demodBn);
```

Converts each 5-bit binary sequence into a decimal value.

Example:

```
00110 → 6
```

```
11111 → 31
```

Convert Custom Encoding to ASCII

```
ascii(ascii >= 6 & ascii <= 31) = ascii(ascii >= 6 & ascii <= 31) + 91;
```

★ Most Important Line

The project uses a custom 5-bit character encoding.

This line converts values 6–31 back into standard ASCII letters:

```
6 → a  
7 → b  
...  
31 → z
```

Convert Special Characters

```
ascii(ascii == 0) = 32;
```

Maps:

```
0 → Space
```

```
ascii(ascii == 1) = 44;
```

Maps:

```
1 → ,
```

```
ascii(ascii == 2) = 46;
```

Maps:

```
2 → .
```

```
ascii(ascii == 3) = 39;
```

Maps:

```
3 → ' 
```

```
ascii(ascii == 4) = 10;  
ascii(ascii == 5) = 13;
```

Maps:

```
4 → Line Feed  
5 → Carriage Return
```

These values preserve the original formatting of the text file.

Convert to Text

```
text = char(ascii);
```

★ Final Output

Converts the ASCII values into readable characters and reconstructs the original message.

Summary

`bin2text.m` converts the filtered QPSK symbols back into readable text. It first extracts the bits carried by each QPSK symbol using threshold detection, groups the bits into 5-bit character codes, converts them into decimal values, and then

maps the custom encoding back to standard ASCII characters. The final result is the recovered text message produced by the decoder.

file2bin(file)-

```
function [x,ascii] = file2bin(file)
% 'file' should be the filename of a simple text file,
% including extension, e.g. foo.txt
% read file contents. Every call to fgets retrieves one line of text.
% the loop concatenates them, making a variable s where every row is a text
% string corresponding to one line of the text.
s=[];
fid=fopen(file,'r');
while (~feof(fid));
s=[s fgets(fid)];
end
as = double(lower(char(s))); % convert to lower case, then to numeric ASCII
code
% find indices that have spaces, line feeds, and so forth, and those that
% have regular characters
lindex = find((as>96) & (as<123)); % lower case letters
spindex = find(as==32);
lfindex = find(as==10);
rindex = find(as==13);
cindex = find(as==44);
pindex = find(as==46);
qindex = find(as==39);
% recode the sequence so everything fits in the range 0:31 and therefore
% into 5 bits. Anything that isn't space, comma, period, apostrophe,
% line feed, return, or 'a' through 'z' is left as a NaN.
asciiseq = NaN ones(1,length(as));
asciiseq(lindex) = as(lindex)-97;
asciiseq(spindex) = 0;
asciiseq(lfindex) = 4;
asciiseq(rindex) = 5;
asciiseq(cindex) = 1;
asciiseq(pindex) = 2;
asciiseq(qindex) = 3;
ascii = NaN ones(1,length(as));
ascii(lindex) = as(lindex);
ascii(spindex) = as(spindex);
ascii(lfindex) = as(lfindex);
ascii(rindex) = as(rindex);
ascii(cindex) = as(cindex);
ascii(pindex) = as(pindex);
ascii(qindex) = as(qindex);
ascii = ascii(find(~isnan(asciiseq)));
% build a new sequence that has the training sequence first, then all
% characters that didn't get left as NaNs, then five spaces.
asciiseq = [0:31 4 5 asciiseq(find(~isnan(asciiseq))) 0 0 0 0 0];
% drop off one of the trailing spaces if need be to make the length even
```

```

l=floor(length(asciiseq)/2)*2;
asciiseq = asciiseq(1:l);
%convert characters to binary strings, then to 2-bit symbols. Each column
%of bb is one pair of bits
bb=reshape(dec2bin(asciiseq)',2,length(asciiseq)*5/2);
%QPSK encoding
x = ((-1).^bb') * [1;j];

```

This file converts a text message into **QPSK symbols** that can be transmitted through the communication channel. It is essentially the opposite of `bin2text.m`.

The process involves:

1. Reading the text file.
2. Converting characters into a custom 5-bit encoding.
3. Adding a training/header sequence.
4. Converting the bits into QPSK symbols.

Overall Functionality

```

Text File
  ↓
Read Characters
  ↓
Custom 5-bit Encoding
  ↓
Add Training Sequence
  ↓
Convert to Binary
  ↓
QPSK Modulation
  ↓
Transmitted Signal

```

Important Code Sections

Read the Text File

```

fid = fopen(file,'r');

while (~feof(fid))

```

```
s = [s fgets(fid)];  
end
```

Reads the entire text file and stores its contents in `s`.

This is the original message that will be transmitted.

Convert to Lowercase ASCII

```
as = double(lower(char(s)));
```

Converts:

- All characters to lowercase
- Characters into their ASCII values

Example:

```
'A' → 'a' → 97  
'B' → 'b' → 98
```

This ensures a consistent character representation.

Identify Supported Characters

```
lindex = find((as>96) & (as<123));
```

Finds all lowercase letters:

```
a-z
```

```
spindex = find(as==32);  
cindex  = find(as==44);  
pindex  = find(as==46);  
qindex  = find(as==39);
```

Finds special characters:

```
Space  
Comma (,)  
Period (.)  
Apostrophe (')
```

Only these characters are supported by the custom encoding scheme.

Create Custom 5-bit Encoding

```
asciiseq(lindex) = as(lindex)-91;
```

★ Important Line

Maps letters into the range:

```
a → 6  
b → 7  
...  
z → 31
```

This allows every character to fit inside 5 bits.

Encode Special Characters

```
asciiseq(spindex) = 0;  
asciiseq(cindex) = 1;  
asciiseq(pindex) = 2;  
asciiseq(qindex) = 3;  
asciiseq(lfindex) = 4;  
asciiseq(rindex) = 5;
```

Custom encoding table:

Value	Character
0	Space
1	,

Value	Character
2	.
3	,
4	Line Feed
5	Carriage Return
6-31	a-z

This custom mapping allows every supported character to be represented using only 5 bits.

Add Training Sequence

```
asciiseq = [0:31 4 5 asciiseq(find(~isnan(asciiseq))) 0 0 0 0 0 0];
```

★ Most Important Line

Adds a known header sequence before the actual message.

Structure:

```
Training Sequence
      +
Message Data
      +
Padding
```

This training sequence is later used by the LMS adaptive filter to learn the channel characteristics.

It is also the reason why:

```
head_signal = 85;
```

must be removed during evaluation.

Ensure Even Length

```
l = floor(length(asciiseq)/2)*2;  
asciiseq = asciiseq(1:l);
```

Makes sure the sequence length is even.

Since QPSK carries 2 bits per symbol, an even number of encoded values simplifies symbol generation.

Convert to Binary

```
bb = reshape(dec2bin(asciiseq)',2,length(asciiseq)*5/2);
```

Converts the 5-bit character codes into a continuous binary stream.

Example:

```
6 → 00110  
7 → 00111
```

The bits are then grouped into pairs because each QPSK symbol carries 2 bits.

QPSK Modulation

```
x = ((-1).^bb') * [1;j];
```

★ Most Important Line in the File

Converts every pair of bits into a QPSK symbol.

Example:

```
00 → ( 1 + j)  
01 → ( 1 - j)  
10 → (-1 + j)  
11 → (-1 - j)
```

This generates the complex QPSK signal that will be transmitted through the channel.

Outputs

```
[x, ascii]
```

x

The generated QPSK symbol sequence.

ascii

The original ASCII values of the message.

Summary

`file2bin.m` prepares the text message for transmission. It reads the input text file, converts supported characters into a compact 5-bit encoding, inserts a known training sequence for LMS filter training, and then performs QPSK modulation to generate the transmitted signal. The resulting QPSK symbols form the input to the communication channel and serve as the starting point of the entire decoding pipeline.

main.m-

```
clear
load gettysburg;
load AllSignals;
V = x20;
message = lms_decode(V);
% 7 9 13
```

This is the **main driver script** of the project. It initializes the environment, loads the test signals, selects a noisy QPSK signal, and starts the LMS decoding process.

Although the file is very short, it serves as the entry point for the entire system.

Overall Functionality

```
Load Test Data
↓
```

```
Select Signal
      ↓
Run LMS Decoder
      ↓
Recover Message
```

Important Code Sections

Clear Workspace

```
clear
```

Removes all existing variables from MATLAB memory.

This ensures that results from previous runs do not affect the current experiment.

Load Original Data

```
load gettysburg;
```

Loads the original Gettysburg message and related data used throughout the project.

This message ultimately serves as the ground truth for evaluation.

Load Test Signals

```
load AllSignals;
```

Loads all previously generated noisy signals from `AllSignals.mat`.

These signals were created by `genSignal.m` under different SNR levels and channel conditions.

Select Test Signal

```
V = x20;
```

★ Most Important Line

Selects the signal that will be decoded.

In this case:

```
x20 → Signal with 20 dB SNR
```

Higher SNR means less noise and generally better decoding performance.

You can replace `x20` with other signals such as:

```
V = x5;  
V = x10;  
V = x10_ti;  
V = x10_tv;
```

to test the LMS decoder under different conditions.

Run LMS Decoder

```
message = lms_decode(V);
```

★ Core Function Call

Passes the selected noisy signal into the LMS decoding pipeline.

This function performs:

1. LMS filter training
2. Noise reduction
3. QPSK symbol recovery
4. Text reconstruction
5. Error-rate evaluation

The recovered message is stored in:

```
message
```

Commented Results

```
% 7 9 13
```

This appears to be a note left by the original developer, possibly indicating filter orders, test cases, or previously observed results.

It does not affect program execution.

Summary

`main.m` is the entry point of the project. It loads the generated test signals, selects one signal for testing, and passes it to the LMS decoder. The decoder then performs adaptive filtering, QPSK recovery, text reconstruction, and performance evaluation. While the file itself is small, it initiates the complete LMS-based QPSK decoding pipeline.